

Let model keep evolving: Incremental learning for encrypted traffic classification

Xiang Li^{a,b}, Jiang Xie^c, Qige Song^c, Yafei Sang^{a,*}, Yongzheng Zhang^d, Shuhao Li^c,
Tianning Zang^{a,b}

^a Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China

^b School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China

^c Zhongguancun Laboratory, Beijing, China

^d China Assets Cybersecurity Technology CO., Ltd., Beijing, China

ARTICLE INFO

Keywords:

Encrypted traffic classification

Evolve

Incremental learning

Multi-view sequences

Cross-view information

Exemplar selection

ABSTRACT

Encrypted Traffic Classification (ETC) is valuable for many network management and security solutions as it provides insights into applications active on the network. However, the network environment constantly evolves, and new applications emerge in an endless stream daily, which gradually makes well-trained ETC models ineffective. The conventional approach to adapting new applications is to re-train the models on a re-formed dataset with both pre-existing and new application samples. The major limitation is that requiring redundant computing resources and sufficient storage spaces. In this work, we propose an Incremental Learning (IL) framework based on multi-view sequences fusion, MISS, to keep ETC models evolving with new applications. The key novelty of MISS is three-fold: extract cross-view information from multi-view sequences to capture sufficient knowledge; propose an exemplar selection algorithm from communication patterns to reduce redundant consumption; design a pair of branches from the learnability of parameters to mitigate accuracy loss during evolution. MISS outperforms the existing IL methods of ETC, and the state-of-the-art ETC models using the classic IL framework, on the real-world network traffic datasets, which achieves satisfactory improvements of 11.37%↑ and 1.58%↑. Furthermore, we comprehensively perform incremental experiments to evaluate the evolution ability of MISS, which is able to select representative exemplars of old applications, counteract the adverse effects of homogeneous applications, and keep evolving with unknown applications.

1. Introduction

Traffic classification is the process of identifying network applications and classifying the corresponding traffic, which is considered to be the most fundamental functionality in QoS, QoE, and network management (Chen et al., 2021; Zhao et al., 2021). In recent years, we have witnessed the rapid development of new network technologies and mobile ecosystems, accompanied by one key evolution where the traffic is encrypted from plaintext in nature. A large number of websites and applications utilize TLS technology for privacy protection and secure communication (Liu et al., 2017). According to the Annual Report of Let's Encrypt (Group, 2020), HTTPS page loads reach 84% globally. Such a trend of encryption dramatically reduces the available plaintext

information, disabling the traditional methods that rely on plaintext, such as DPI (Deep Packet Inspection).

Several solutions for Encrypted Traffic Classification (ETC) have been proposed by researchers (Liu et al., 2019; Shen et al., 2017; Zheng et al., 2020), which extract classifiable sequence features (mostly side-channel information) from their standpoint. However, most of these only work in close-world environments, namely, training ETC models with a fixed number of applications. *Obviously, these models face a practical problem: when deployed in an open network environment, they gradually lose their classification ability due to the growing number of new applications.* An intuitive explanation is shown in Fig. 1. The old model $\mathcal{M}$ in the 0th phase, trained on TikTok, Amap, and Twitter, couldn't classify Alipay, Taobao, and Pingduoduo in the 1th phase. The most straightfor-

* Corresponding author.

E-mail addresses: lixiang1@iie.ac.cn (X. Li), sangyf793@gmail.com (Y. Sang).

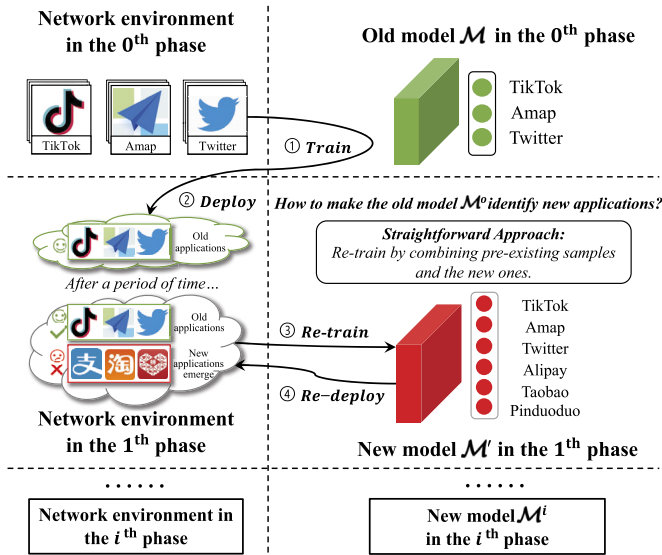


Fig. 1. The old model trained on TikTok, Amap, and Twitter in the 0th phase is unable to perform classification effectively in the 1st phase because Alipay, Taobao, and Pingduoduo are added.

ward approach for alleviating this issue is to re-train a new model $\mathcal{M}'$ by combining pre-existing samples and the new ones. However, frequently re-train the model from scratch is redundant, time-consuming, and costly (Xu et al., 2020b; Zhu et al., 2021).

Natural learning systems such as humans inherently work in an incremental evolution as concepts increase over time. An ideal ETC model should also keep evolving incrementally like natural learning systems without training from scratch when new applications are considered. However, the model evolution inevitably forgets knowledge acquired from old applications, since the new optimizer will override the parameters tuned by the old. It is not a wise choice by replaying the entire pre-existing samples to regain forgotten knowledge. This imperfection, as mentioned above, can be reflected in two parts, for one thing, this is redundant work while training resource is wasted, and for another, the storage space is limited for entirely storing increasing samples.

In this paper, we propose an Incremental Learning (IL) framework based on Multi-view Sequences FuSion, MISS,¹ to keep ETC models evolving with new applications. The key novelty of MISS is three-fold:

- ❶ Evolution is the extension of the old model. How to get an outstanding old model for evolution?
- ❷ Storage is limited. How to save as few pre-existing samples as possible to reduce redundant storage?
- ❸ Forgetting is inevitable. How to reduce the forgetting of old knowledge when learning new applications?

To answer ❶, MISS extracts cross-view information based on multi-view sequences to enhance the initial classification ability of old model $\mathcal{M}$, and to provide enough information that can be inherited stably by the next phase. In response to ❷, an exemplar selection algorithm from the communication patterns is proposed to reduce the redundant storage of pre-existing samples. For ❸, a pair of plastic and stable branches of new model $\mathcal{M}'$ are designed: one for learning new applications, and the other for maintaining old knowledge, to mitigate accuracy loss in evolution.

Overall, MISS constructs an old model $\mathcal{M}$ that captures sufficient knowledge by extracting cross-view information based on multi-view sequences. During the evolution process, model $\mathcal{M}$ initializes the plastic

branch of model $\mathcal{M}'$ to preserve learnability and then freezes itself as the stable branch to maintain old knowledge. All learnable parameters of model $\mathcal{M}'$ are modified on a handful of pre-existing exemplars and all new samples.

Our contribution can be briefly summarized as:

- We design an IL framework, MISS, to keep ETC models classifying new applications without training from scratch. In addition, a selection algorithm of representative exemplars is provided for ETC tasks to avoid redundant storage.
- We build a pure TLS-encrypted traffic dataset, named PURE-TLS, which contains 100 applications with 166,939 TLS flows. Evaluations on PURE-TLS and three public datasets prove that MISS has great classification accuracy and evolution ability, compared with the existing IL methods of ETC and the state-of-the-art (SOTA) ETC models using the classic IL framework.
- We conduct extensive incremental experiments to evaluate the evolution ability of MISS, which is able to select more representative exemplars, counteract the adverse effects of homogeneous applications, and keep evolving with unknown applications.

Roadmap: The rest of the paper is organized as follows. Section 2 summarizes the existing ETC and IL methods. Section 3 introduces the basic definitions and concepts. Details of the MISS framework are provided in Section 4. Section 5 shows the experiments and evaluations. Section 6 and Section 7 put the discussion and conclusion.

2. Related work

We focus on two research areas in terms of related work: *Encryption Traffic Classification* and *Incremental Learning*. Firstly, the past decade has seen extensive research exploring encrypted traffic. We focus on the utility of statistical and sequence features for the ETC. IL has continuous research in Computer Vision (CV), and part of the work has appeared in ETC. We elaborate on the existing work on IL from two perspectives: *IL methods for CV* and *IL methods for ETC*.

2.1. Encryption traffic classification

ETC tasks are based on the characteristics of encrypted traffic into different classes, such as applications, protocols or services, and have important applications in network management, security and personalized recommendations.

Early on, researchers propose approaches based on port number and data packet inspection, where traffic is classified by examining the traffic port number or packet header information (Bujlow et al., 2015; Finsterbusch et al., 2013). However, this approach is susceptible to interference from techniques such as port masquerading, packet obfuscation and encryption. Then, machine learning-based approaches are increasingly being employed for ETC tasks for their automated modeling and analysis capabilities (Velan et al., 2015). Researchers have proposed several approaches combining statistical features and machine learning algorithms to solve ETC tasks, e.g., (Aceto et al., 2017; Liu et al., 2012; Taylor et al., 2017). Encrypted applications leak information about dependencies or transfers between data messages, i.e., sequence features, which are proven more efficient and capture richer content and contextual information (Liu et al., 2019; Wang et al., 2017a, 2020). Korczynski et al. (Korczynski and Duda, 2014) proposed to represent the TLS content type sequence with a markov transformation matrix to recognize the encrypted flow of applications. Rebuffi et al. (Shen et al., 2017) propose an attribute-aware ETC method based on the second-order markov chains that incorporating the attribute bigrams into the homogeneous markov chains.

Currently, deep learning-based approaches with excellent performance on ETC tasks, which draw on deep learning techniques from fields such as CV (e.g., convolutional neural networks, recurrent neural

¹ For readability not meaning, we shuffle the order to form a word.

networks, attention mechanisms), encode and classify raw traffic data or high-level representations directly, without decomposing the problem into sub-problems of feature selection and classification (Rezaei and Liu, 2019; Wang et al., 2019). Deep learning is a breakthrough compared to other methods, being able to automatically extract complex features from traffic, eliminating the need for expert knowledge to select features, and having a significant learning capability. Zheng et al. (Zheng et al., 2020) built an autoencoder model to restore the packet length sequence. Lin et al. (Lin et al., 2022) adopted the bidirectional encoder representations transformer to extract features from payloads byte sequence. Fu et al. (Fu et al., 2016) extracted packet length and arrival time sequences to build a hidden markov model, which in turn enhances the intrinsic richness of the model.

Most of the existing ETC methods are based on a single or fewer sequence features, which are only highly generalized for one aspect of the encrypted traffic and easily influenced by the network environment. Exploring the sufficient knowledge and association relationship from multiple sequence views to get an outstanding ETC model is one of the main contributions of this work.

2.2. Incremental learning

2.2.1. IL methods for CV

IL aims to learn efficient models gradually in a series of incremental phases, each with new classes (Liu et al., 2021; Zhang et al., 2021). Recent methods in CV have focused on how to avoid forgetting old knowledge. Rebuffi et al. (Rebuffi et al., 2017) used a fixed-size storage space to store the exemplars of old classes, which replayed to reduce forgetting. Ostapenko et al. (Ostapenko et al., 2019) proposed a method that synthesizes fake samples of the old tasks to balance the number of exemplars between old and new classes. Rusu et al. (Rusu et al., 2016) proposed the progressive network, which dedicated partially invariant model parameters in different incremental phases. Hou et al. (Hou et al., 2019) introduced a set of regularization terms to mitigate the adverse effects of data imbalance between old and new classes. Douillard et al. (Douillard et al., 2020) introduced PODNet, which uses varied pooling dimensions in knowledge distillation to bridge feature representations between consecutive models. Yan et al. (Yan et al., 2021) introduced DER, which learned new features through the addition of a new network at each incremental step, and then utilized the concatenated old and new features for the classification. Kang et al. (Kang et al., 2022) proposed a class-incremental learning method that uses knowledge distillation with adaptive feature consolidation to prevent the model from forgetting the knowledge of old classes while learning new ones.

2.2.2. IL methods for ETC

IL for ETC also attracts researchers to explore. Sun et al. (Sun et al., 2017) utilized a strategy known as one vs rest (OvR) to expand the binary SVM model for identifying new applications. This approach was also adopted by Chen et al. (Chen et al., 2021), who used OvR in conjunction with neural network classifiers to classify encrypted traffic incrementally. However, the scalability of the OvR approach is limited, mainly when a large number of new applications are introduced. In response to this limitation, alternative approaches have been explored. Bovenzi et al. (Bovenzi et al., 2021) investigated using the *iCarl* framework for the incremental classification of encrypted traffic. Wu et al. (Wu et al., 2022) proposed using knowledge distillation and bias correction to mitigate the issue of catastrophic forgetting in standard CNNs when new applications are introduced. Li et al. (Li et al., 2020) introduced an incremental classification framework based on a multiclass support vector machine that can add new classification planes and update all old planes for better classification quality. Lee et al. (Lee et al., 2020) employed an incremental SVM with a Stochastic Gradient Descent algorithm to accomplish the encrypted malware traffic detection task. Zhu et al. (Zhu et al., 2022) proposed a federated incremental learning scheme, Fed-SOINN, suited to Internet of Things (IoT) traffic

classification to handle tight bandwidth resources and unstable channels efficiently.

Encrypted flows have richer background information and a priori information compared with images. IL for ETC can be achieved not only by balancing old and new knowledge with IL frameworks, but also be guided to completion by some extra additional information. However, the existing work only performs framework migration from CV to ETC. In contrast to them, we systematically design an IL framework applicable to ETC that exploits traffic-specific information.

3. Preliminaries

3.1. IL problem definition of ETC

IL differs from the conventional classification setting in three aspects (Liu et al., 2020): 1) the training data come in as a stream where the samples of different applications occur in different phases; 2) in each phase, the IL classifier is expected to provide competitive performance for all seen applications so far; 3) the machine storage is limited, so it is impossible to save all data to replay the network training. Based on the above conditions, we define the IL problem of ETC.

Suppose there are $N+1$ phases, i.e., one initial phase,² and N incremental phases, of the ETC process. In the 0th (initial) phase, we build an initial model $\mathcal{M}^0$ on dataset $\mathcal{D}^0$, which consists of $\{c_0, \dots, c_{n_0}\}$ (shortly as $\mathcal{C}^0$) applications. In the i^{th} (incremental) phase, model $\mathcal{M}^{i-1}$ is unable to perform effective classification when $\mathcal{D}^i$ consisting of $\mathcal{C}^i$ applications arrives. Due to the storage limitation, we can't keep the entire previous $\{\mathcal{D}^0, \dots, \mathcal{D}^{i-1}\}$ (as $\mathcal{D}^{0:i-1}$), but instead, we select and store a handful of exemplars, namely $\lambda^{0:i-1}$, as a replacement with $|\lambda| \ll |\mathcal{D}|$. All that needs to be done is to build the $\mathcal{M}^i$ initialized by $\mathcal{M}^{i-1}$ using the new $\mathcal{D}^i$ and the old $\lambda^{0:i-1}$ loading from storage, in order to achieve a competitive classification performance on the test data (unseen) of all applications observed so far.

3.2. Descriptions of multi-view sequence features

In this work, the basic unit of ETC is a flow, which is a collection of packets exchanged between a pair of endpoints for the purpose of inter-process communication across the Internet (Wang et al., 2013). And the flow is bi-directional, comprising a pair of unidirectional sub-flows going in opposite directions, in which the packets sent in each direction can be identified by a unique 5-tuple $\{\text{source IP, destination IP, source port, destination port, protocol}\}$. In order to apply machine/deep learning classification techniques, the flow is further described by its measurements on a predetermined sequence feature set (Millar et al., 2021; Rezaei and Liu, 2019). In other words, each flow is represented by a vector set $\{x_v\}_{v=1}^V$, which can be regarded as a data instance observed from V views.

Specifically, we define the sequence feature set from the perspective of multiple granularity views that provide sufficient information for the inheritance of evolution, as shown in Fig. 2. We measure the *inter-arrival time sequence feature* of packets based on a sliding window, along with the *packet length sequence feature* from the whole packet, and the *content type* and *payload byte sequence features* from the detailed TLS header and TLS payload, respectively (Table 1). Depending on the existing work experience and sensitivity analysis (Lin et al., 2022; Lotfollahi et al., 2020), We extract the sequences of $n=32$ packets, and based on that, only the first $m=128$ bytes of payload are extracted.

3.2.1. Inter-arrival time sequence feature

We measure the inter-arrival time in seconds between each packet of a flow. To reduce the interference of network problems (e.g., network

² For new applications/models, only the 0th phase is initial, and the others are called old.

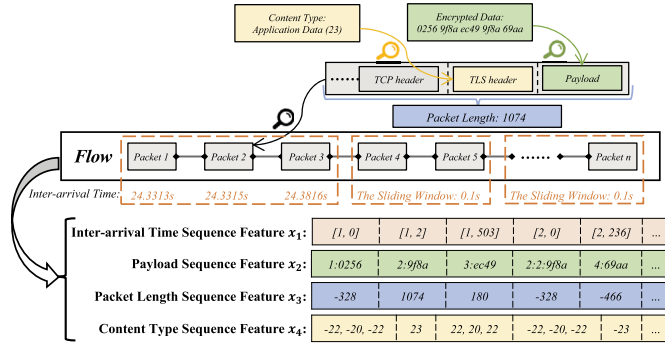


Fig. 2. Extract the inter-arrival time, packet length, content type, and payload byte sequence features from multiple granularity views, respectively.

Table 1
List of sequence features.

Symbol	Description
x_1	Inter-arrival time sequence feature.
x_2	Payload sequence feature.
x_3	Packet length sequence feature.
x_4	Content type sequence feature.

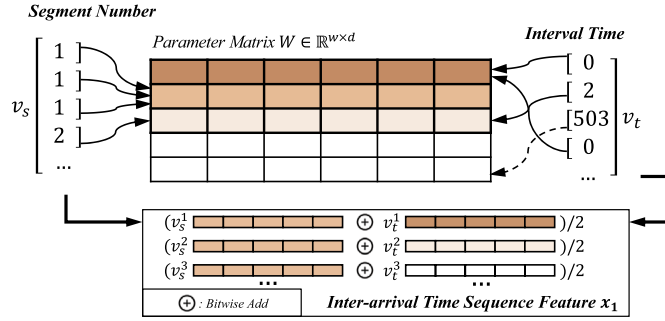


Fig. 3. Inter-arrival time sequence feature with embedding.

jitter), we split the flow into multiple segments based on a sliding window win (e.g., 0.1 s), and extend the inter-arrival time, e.g., [24.3313, 24.3315, 24.3816, 24.4852, 24.5088...], into a 2-tuple $x_1 = \{v_s, v_t\}$, e.g., $\{[0, 0], [0, 122], [0, 503], [1, 0], [1, 236] \dots\}$, one of the segment number, the other of the interval time from the first packet in its segment. To normalize the extended inter-arrival time into a vector, we perform word embedding of $\{v_s, v_t\}$ through the parameter matrix $W \in \mathbb{R}^{w \times d}$, which transforms the discrete input sequence $\{v_s, v_t\} \in \mathbb{R}^{n \times 1}$ into a high-dimensional vector $\{v_s, v_t\} \in \mathbb{R}^{n \times d}$, where w denotes the size of W , and d is the dimension. Then they are added and averaged together bitwise, as shown in Fig. 3.

3.2.2. Payload sequence feature

Although TLS encrypts the payload, some side-channel information can still be leaked (Sherry et al., 2015). To extract more context-sensitive information, we divide the payload into two-byte chunks. For instance, the payload [0256f8aec499f8a69aa...] is divided into [0256, 9f8a, ec49, 9f8a, 69aa...]. Following this, we recode it using a 1-gram dictionary. An exemplar of this is {0256:0, 9f8a:1, ec49:2, 69aa:3...}. This allows us to form the payload feature vector, for instance [0, 1, 2, 1, 3...], which is then embedded as shown in Fig. 4.

3.2.3. Packet length sequence feature

We measure the packet length in bytes. Due to the bi-directional nature of the flow, we further use positive and negative to denote directionality. Specifically, we use positive to indicate the packet sent from

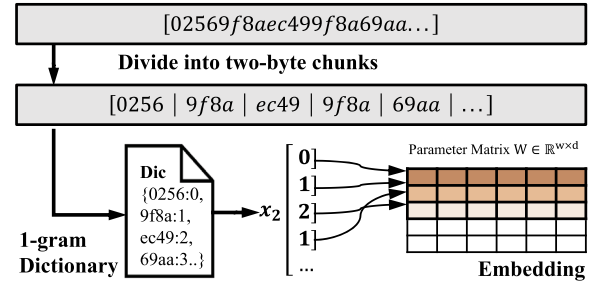


Fig. 4. Payload sequence feature with embedding.

the client to the server and negative to indicate sent from the server to the client, e.g., [-328, 1074, 180, -328, -466...]. To process this raw data for our learning model, we also employ a 1-gram dictionary for re-encoding and perform subsequent embedding operations. This procedure transforms the sequence of packet lengths (including $\pm$) into a series of numbers, which are then embedded, just like the payload sequence feature operation.

3.2.4. Content type sequence feature

The content type is a flag in the TLS header that indicates the semantic information, e.g., *handshake:22, application data:23*, of the packet encrypted using TLS (Rescorla and Modadugu, 2012). We also use positive and negative to denote directionality, e.g., [-22, -20, -22, 23, 22, 20, 22...]. We again utilize a 1-gram dictionary for re-encoding and performing embedding operations.

Especially to be mentioned, the sequence feature set of MISS is scalable due to the fact that feature extraction is performed independently for each view. It is recommended to use more sequence views (e.g., calculated window size) to ensure the diversity and complementarity of application knowledge.

4. The proposed MISS

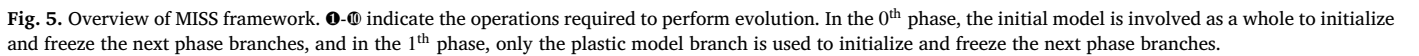
In this section, we detail the proposed MISS. The overall workflow of MISS is depicted in Fig. 5. We describe the initial model in the 0th phase in detail, where the sequence features extraction, interaction, and fusion processes between different views are represented. Then, the evolution in the i^{th} phase is elaborated from the data and model structure perspectives. The optimization strategy is presented finally.

4.1. Solution overview

We construct the solution as a two-stage process, i.e., the initial model in the 0th phase and the evolution in the i^{th} phase, according to the problem definition: the aim of the first stage is to design an outstanding initial model in the 0th (initial) phase, and the second is to maintain learning new applications and reduce the forgetting of the old in the i^{th} (incremental) phase.

For an evolvable ETC model, the initial classification ability is the key to determining the subsequent ability, which can provide accurate and sufficient information to be inherited stably by the next phase. In the 0th phase, the initial model structure mainly consists of three components: multi-view sequence features extraction, cross-view information generation, and multi-view adaptive fusion, which seamlessly embeds various view sequence features, generates stable cross-view information, and makes joint decisions by adaptive fusion strategy to improve the classification performance.

In the evolution process, forgetting is unavoidable because the parameters adjusted by the old applications will be overwritten when learning the new. From the problem definition, it is clearly unwise to store the entire previous samples to regain forgotten knowledge. In the i^{th} phase, we design the evolution from the data and model structure



A list of abbreviations is presented in Table 2 to facilitate understanding of the notations adopted in what follows.

4.2.1. Multi-view sequence features extraction

where $y_v \in \mathbb{R}^d$, and the parameters of each view are learned independently in parallel.

$$C_v = \{\mathbb{E}[y_v, y_{\tilde{v}}^\top]\}_{\tilde{v}=\{1, \dots, V\} \setminus v} \quad (2)$$

List of abbreviations.

where $|$ is the splicing operation, $W_\phi \in \mathbb{R}^{d \times d^2}$ and $b_\phi \in \mathbb{R}^d$ which parameters are learned and shared across V views.

Sequential features from different views complement classification tasks, yet the precise contribution of each feature is uncertain. To address this issue, we assemble all view feature matrices into multi-channels to form a 3D matrix X and input them into a *multi-layer perceptrons* (MLPs) (Tolstikhin et al., 2021). This network, composed of fully-connected layers and activation functions, adaptively captures the

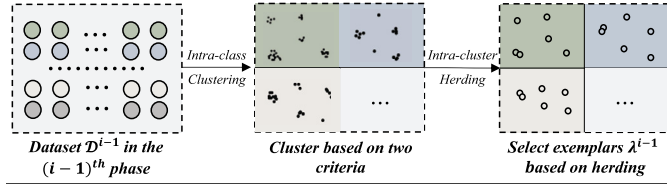


Fig. 6. Representative exemplar selection for D^{i-1} . The selected exemplars will be stored after each phase completion.

intrinsic relationships between different views. Specifically, the operations are expressed as:

$$\begin{aligned} X &= [Y_1 \mid Y_2 \mid \dots \mid Y_V], \\ U &= X + F_2 \sigma(F_1(LN(X))), \\ Y &= U + F_4 \sigma(F_3(LN(U))) \end{aligned} \quad (4)$$

where LN means the LayerNorm operation, F_{1-4} denote the fully-connected layers, and σ is an element-wise nonlinearity (e.g., GELU (Hendrycks and Gimpel, 2016)). The resultant multi-view feature representation, Y (the output of MLPs), is then fed into the classification layer to predict the label, which is optimized by the cross-entropy loss.

4.3. The evolution in the i^{th} phase

4.3.1. Representative exemplar selection

The number of exemplars $\lambda^{0:i-1}$ is significantly smaller than $D^{0:i-1}$. Existing IL methods (Castro et al., 2018; Hou et al., 2019; Liu et al., 2020) are always based on the assumption that the model trained on a handful of exemplars also minimizes the loss of all old samples. In order to achieve this in ETC tasks, we split the exemplar selection into two steps: *Intra-class Clustering* and *Intra-cluster Herding*. Initially, for each old class, we identify all its communication pattern clusters through *Intra-class Clustering*. This step ensures that we capture the diverse communication patterns inherent within each class. Subsequently, during the *Intra-cluster Herding* phase, we select the most representative exemplars from each identified communication pattern cluster. This meticulous selection approach guarantees that the chosen exemplars comprehensively and accurately reflect the intrinsic structure and distribution characteristics of each class, thereby providing a more holistic and representative exemplar set for model evolution, as illustrated in Fig. 6. Algorithm 1 further details this process.

Intra-class Clustering. Applications typically consist of various modules, each communicating with external servers or services to fulfill their functionalities. Such communications are often associated with specific network destinations, such as server IP addresses and ports. These communication destinations tend to remain relatively stable over time, providing clues to the inherent communication patterns of applications. To capture these patterns, we cluster the samples of application c in D^{i-1} based on the following two criteria:

- Samples containing identical {destination IP, destination port}-tuples.
- Samples containing identical/similar TLS certificates.

This clustering approach is grounded in a deep understanding of real-world communication patterns exhibited by applications. It's worth noting that while identical {destination IP, destination port}-tuples might map to different services due to mechanisms like CDNs or load balancers, our objective isn't to precisely identify specific service attributes of the traffic. Instead, we aim to select representative exemplars that effectively capture the unique communication patterns of applications. Consequently, choosing representative exemplars from consistent CDN traffic or load balancers aids the model in gaining a deeper understanding of these patterns. Additionally, considering that the number

of samples in some clusters might be less than S or that some samples might not be clustered, we consolidate these samples into a comprehensive cluster.

Intra-cluster Herding. For each cluster of application c , we acknowledge its importance in the exemplar selection process. To better capture the characteristics of cluster z , we first compute the average feature vector $\bar{\eta}$ of it using model M_{i-1} . Subsequently, we employ the *herding algorithm* (Welling, 2009), an iterative sample selection method designed to pick samples that best represent the overall data distribution. Unlike methods that rely on randomness, the *herding algorithm* iteratively selects exemplars closest to the current average feature vector, ensuring that the chosen exemplar set maximally reflects the original data's feature distribution. To ensure that all applications are treated equitably during the selection process, we determine the value of K , which represents the number of representative exemplars selected from each cluster. Specifically, we have:

$$K = \text{ceil} \left(\frac{\bar{C} \times \mathcal{T} \% \times \text{len}(z)}{\text{len}(c)} \right) \quad (5)$$

where $K \geq 1$, $\text{ceil}(\cdot)$ is a mathematical function that rounds a number up to the nearest integer, $\bar{C}$ denotes the average sample count of the class, $\mathcal{T} \%$ is the exemplar retention rate, and $\text{len}(\cdot)$ is a quantity measurement function. Through the selection strategy of the herding algorithm, we ensure that the average feature vector of the representative exemplars closely approximates the average feature vector of all samples within the application. Moreover, by treating all applications uniformly and considering all clusters for each application, we guarantee that the data's diversity and completeness are fully taken into account. This strategy, while retaining a subset of samples as exemplars, minimizes the risk of altering or eliminating the relationships and differences between classes.

Algorithm 1: Representative Exemplar Selection.

Input : Dataset D^{i-1} of applications C^{i-1}
Destination information Λ^{i-1} of D^{i-1}
Average sample amount $\bar{C}$ (per class)
Minimum S and top $\mathcal{T} \%$
Require: $(i-1)^{\text{th}}$ phase model M^{i-1}
Clustering algorithm Ψ
Index mapping function ξ

for c in C^{i-1} **do**
 $\mathcal{Z} \leftarrow \Psi(\Lambda_c^{i-1}, S)$ // Intra-class Clustering
 for z in $\mathcal{Z}$ **do**
 $\bar{\eta} \leftarrow \text{mean}(\sum_{m \in z} M^{i-1}(\xi(m, D^{i-1})))$
 $K = \text{ceil}(\bar{C} \times \mathcal{T} \% \times \text{len}(z) / \text{len}(c))$
 for $j = 1, \dots, K$ **do**
 $p_j \leftarrow \arg\min_{m \in z} \|\bar{\eta} - \frac{1}{j} \times [M^{i-1}(\xi(m, D^{i-1})) + \sum_{k=1}^{j-1} M^{i-1}(\xi(p_k, D^{i-1}))]\|$
 // Intra-cluster Herding
 $P_z \leftarrow \{p_1, \dots, p_K\}$
 $\mathcal{P}_c \leftarrow \{P_z\}_{z \in \mathcal{Z}}$
 $\lambda^i \leftarrow \{P_c\}_{c \in C^{i-1}}$
if $i = 1$ **then**
 Output : Exemplar set λ^{i-1}
else
 $\lambda^{0:i-1} \leftarrow \{\lambda^0, \dots, \lambda^{i-2}\} + \lambda^{i-1}$
 Output : Exemplar set $\lambda^{0:i-1}$

4.3.2. Plastic and stable branches

In the i^{th} phase, our model deploys a pair of branches, namely, the Plastic Branch and the Stable Branch, distinguished by the different learnability of their parameters. The Plastic Branch, initialized by M^{i-1} (or the entire initial model M^0), serves to adapt and learn from new data. All parameters in this branch are learnable and updated at each stage, allowing the model to continuously optimize with the introduction of new information. The Stable Branch, on the other hand, acts as the knowledge store of our model. After the Plastic Branch initializes the next phase, the current model freezes itself as the Stable Branch,

only preserving the neuron-level scaling weights to maintain the structural pattern within the neuron (Liu et al., 2021; Sun et al., 2019). This process ensures the retention of learned knowledge, acting as a protective mechanism against the catastrophic forgetting problem in the continuous learning process of deep learning models. By re-training all learnable parameters of both branches on the exemplars $\lambda^{0:i-1}$ and new samples D^i , we strike a balance between adaptation to new data and preservation of previous knowledge. Given a flow instance, we compute the plastic vector Y^p and the stable vector Y^s , through the plastic and stable branches, respectively. Let ϵ^p and ϵ^s denote the aggregation weights, and the weighted sum of Y^p and Y^s can be derived as,

$$\eta = \epsilon^p \cdot Y^p + \epsilon^s \cdot Y^s \quad (6)$$

where ϵ^p and ϵ^s are learnable parameters, and we describe the optimization strategy in detail in Section 4.4.

4.3.3. Classification layer with distillation loss

As we introduce new classes, we strategically discard the existing classification layer. This allows us to introduce a new layer, tailored to accommodate both new and old classes. This approach ensures our model remains adaptable to new classes without sacrificing its proficiency with the old ones. The new classification layer is modified to combine the distillation loss (Hinton et al., 2015) and the cross-entropy loss, which allows remembering the old applications while learning the decision boundaries for the new. The distillation loss $\mathcal{L}_d$ encourages the $\mathcal{M}^i$ and previous $\mathcal{M}^{i-1}$ to maintain the same prediction ability on old applications, formulated as,

$$\mathcal{L}_d(\mathcal{M}^i; \mathcal{M}^{i-1}; \eta) = - \sum_{m=1}^M \hat{\pi}_m(\eta) \log \pi_m(\eta) \quad (7)$$

$$\hat{\pi}_m(\eta) = \frac{e^{\hat{p}_m(\eta)/t}}{\sum_{j=1}^M e^{\hat{p}_j(\eta)/t}}, \quad \pi_m(\eta) = \frac{e^{p_m(\eta)/t}}{\sum_{j=1}^M e^{p_j(\eta)/t}}$$

where M is the number of applications in $\lambda^{0:i-1}$, $p_m(x)$ and $\hat{p}_m(x)$ denote the prediction logits of the m^{th} application from $\mathcal{M}^i$ and $\mathcal{M}^{i-1}$, and t is a temperature scalar set to be greater than 1 to assign larger weights to smaller values. The cross-entropy loss $\mathcal{L}_c$ still is used as the classification loss, formulated as,

$$\mathcal{L}_c(\mathcal{M}^i; \eta) = - \sum_{m=1}^{M+M'} \nabla_{\tilde{y}=m} \log p_m(\eta) \quad (8)$$

where M' is the number of applications in D^i , $\tilde{y}$ is the ground-truth label, and $\nabla_{\tilde{y}=m}$ is an indicator function. As mentioned earlier, we only calculate $\mathcal{L}_c$ in the 0th phase, and jointly calculate $\mathcal{L}_d$ and $\mathcal{L}_c$ to add up in the other phases.

4.4. The optimization strategy

In each incremental phase, we optimize two groups of parameters in MISS:

- The learnable parameters of both branches.
- The aggregation weights $[\epsilon^p, \epsilon^s]$ of branches' output.

The former is for network parameters, and the latter is for hyperparameters. The aim of the i^{th} phase is to learn the optimal branches' parameters and $[\epsilon^p, \epsilon^s]$ that minimize the loss on all training samples seen so far. Based on this objective, we formulate the overall optimization process as a bilevel optimization program (BOP) (Goodfellow et al., 2014; Liu et al., 2021):

- The learnable parameters of both branches are trained using the aggregation weights $[\epsilon^p, \epsilon^s]$ as hyperparameters.

- The aggregation weights $[\epsilon^p, \epsilon^s]$ can be updated by mini-batches of a balanced dataset when the learnable parameters are temporarily fixed.

In this way, the optimality of learnable parameters imposes a constraint on the aggregation weights and vice versa. Algorithm 2 describes the optimization strategy.

Algorithm 2: The Optimization Strategy.

Input : D^i and exemplars $\lambda^{0:i-1}$
Require: $(i-1)^{\text{th}}$ phase model $\mathcal{M}^{i-1}$
 Get D^i and load $\lambda^{0:i-1}$
 Select exemplars λ^i from D^i by Algorithm 1 with $\mathcal{M}^{i-1}$
 Initialize and freeze $\mathcal{M}^i$ with $\mathcal{M}^{i-1}$
for epochs **do**
 for batches in $\lambda^{0:i-1} \cup D^i$ **do**
 Train parameters of $\mathcal{M}^{i-1}$ on $\lambda^{0:i-1} \cup D^i$
 for mini-batches in $\lambda^{0:i-1} \cup \lambda^i$ **do**
 Train $[\epsilon^p, \epsilon^s]$ on $\lambda^{0:i-1} \cup \lambda^i$
 Update exemplar λ^i from D^i by Algorithm 1 with $\mathcal{M}^i$

The balanced dataset, pivotal for our optimization process, is used in mini-batches. It's carefully assembled by combining representative exemplars of new classes, which are initially chosen via our proposed exemplar selection algorithm and based on the representations from the preceding model $\mathcal{M}^{i-1}$, with exemplars from both old $\lambda^{0:i-1}$ and new classes λ^i . This ensures an even distribution of samples from both sets of classes, effectively reducing class-specific bias and optimizing for a more precise $[\epsilon^p, \epsilon^s]$.

As we transition to the current phase model $\mathcal{M}^i$, it becomes crucial to reselect exemplars for the new classes according to the updated representations. This ensures that the new class exemplars align with the latest model representations, making the selected samples more indicative of the exemplars required by the current model.

5. Experiments

In this section, we are dedicated to evaluating the performance of MISS. We first introduce the public datasets and a newly proposed dataset used for conducting comprehensive experiments (Section 5.1), and then explain the experimental settings (Section 5.2). After that, we thoroughly compare MISS with the existing IL methods of ETC, and the SOTA ETC models using classic IL framework, in terms of classification accuracy and evolution ability (Section 5.3). Finally, we comprehensively evaluate the effect of different increment settings on the evolution ability of MISS (Section 5.4). Additionally, our experiments were implemented with Pytorch 1.10.0, conducted with NVIDIA 3090 GPUs.

5.1. Dataset description

To comprehensively validate the performance of the MISS framework, we conduct experiments on three public datasets and one newly proposed dataset, as shown in Table 3.

5.1.1. Cross-platform dataset

The *Cross-Platform (iOS)* (Ren et al., 2019) and *Cross-Platform (Android)* (Ren et al., 2019) collect most popular applications for iOS and Android from each country (US, China, and India), of which randomly select 190 and 200 applications for experiments, respectively. These datasets with long-tail data distribution over all applications.

5.1.2. USTC-TFC-2016 dataset

Encrypted malicious traffic is one of the Internet security threats. The *USTC-TFC-2016* (Wang et al., 2017b) is a collection of encrypted traffic consisting of malware and benign applications, containing 10 benign and 10 malicious.

Table 3

The Statistical Information and Incremental Settings of Datasets. Total means the total number of applications, and Half means the half (or close to half) of the total. N means the number of incremental phases of the dataset, and C means the number of new applications per increment phase.

Dataset	Flow	Total	Half	N Phases	C Classes
Cross-Platform (iOS)	23,987	190	90	[1, 10, 25]	[100, 10, 4]
Cross-Platform (Android)	34,642	200	100	[1, 10, 25]	[100, 10, 4]
USTC-TFC-2016	28,516	20	10	[1, 5]	[10, 2]
PURE-TLS	166,939	100	50	[1, 10, 25]	[50, 5, 2]

5.1.3. PURE-TLS dataset

Most existing public network traffic datasets mix encrypted TLS traffic flows with vast amounts of unencrypted TCP/UDP flows. However, over 90% of today's network traffic is encrypted (Google, 2022). To analyze pure encrypted TLS traffic, we collect 100 applications using Monkeyrunner (Chang et al., 2016) to constitute the *PURE-TLS* dataset, as seen details in Table 4. For the construction of *PURE-TLS*, we install the target applications and the necessary system applications on a rooted Google Pixel3 phone, and then connect to a computer WIFI hotspot to produce the network traffic. Based on PCAPDroid,³ we capture the traffic with application labels directly from the network interface of the Android device. PCAPDroid will add a fake Ethernet header containing the application name to the packet to encapsulate the original IP header and data during traffic capture. In addition, to further ensure the authenticity of data and the accuracy of annotations, we simultaneously employ Wireshark to capture hotspot data on the computer side, and verify the relationship between the Wireshark raw streams and the application labels in PCAPDroid with the five-tuple and timestamp (Aceto et al., 2019a). Finally, we collect and compile a complete dataset of pure encrypted traffic.

To provide a better understanding of the *PURE-TLS* dataset, we illustrate the flow number distribution of the dataset, as presented in Fig. 7. Additionally, a series of the sequence features of the *PURE-TLS* dataset is depicted as heatmaps. Each row corresponds to a different application, while columns represent different sequence positions (packet index), with values averaged across all flows. Upon closely examining the heatmaps, we observe significant variations in flow distribution across different classes. Quantitatively, the most predominant class comprises 7127 flows, while the least populated class has only 98. When analyzing the sequence distribution of average packet lengths, the first 32 packets emerge as crucial for class differentiation. The distribution of average inter-arrival times underscores the dataset's intricacy, revealing unique communication patterns across various applications. Additionally, the packet direction sequence, represented by ± 1 , enhances feature classification. This representation offers deep insights into the nuanced behaviors distinguishing different applications.

5.2. Experimental settings

5.2.1. Traffic preprocessing

We need to perform preprocessing to remove the ethernet header and follow (Lin et al., 2022; Wang et al., 2017a) without feature extraction for the IP header and protocol port of flows in the classification process to avoid the effect of the packet header, which may introduce biased interference for classification in a finite set with strong identification information. The IP and port background knowledge is only used as a priori information (van Ede et al., 2020; Wang et al., 2013) to guide the clustering process when selecting exemplars. We extract the payload byte, arrival time, and packet length sequence views for the three public datasets, adding the content type sequence view for the *PURE-TLS* dataset.

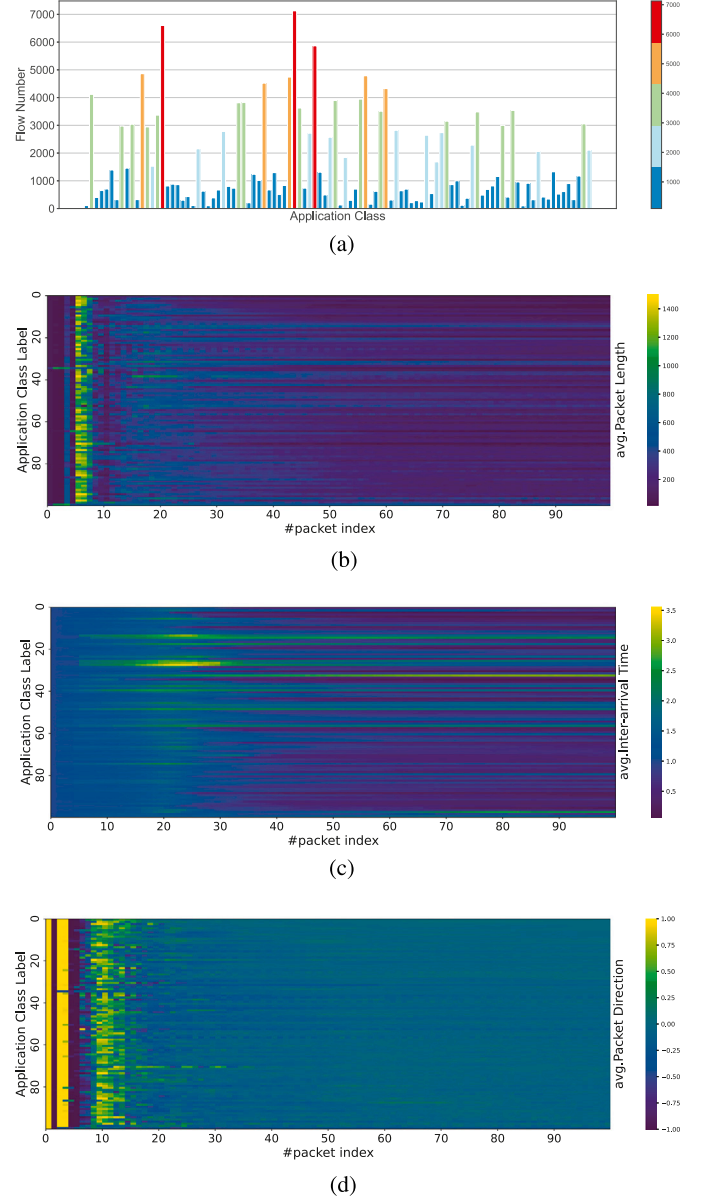


Fig. 7. Visualizations of *PURE-TLS* dataset. (a) Flow number distribution. (b) Sequence distribution (the first 100 packets) of the average packet length of each class. (c) Sequence distribution of the average inter-arrival time of each class. (d) Sequence distribution of the packet direction (± 1) of each class.

5.2.2. Training & testing

For each dataset, approximately half of the total applications are selected to train an initial model, serving as the starting point. The remaining applications are equally divided for N incremental phases, following the recommendations in (Castro et al., 2018; Hou et al., 2019; Rebuffi et al., 2017). To determine the application subset for

³ <https://emanuele-f.github.io/PCAPDroid>.

Table 4
Details of the PURE-TLS Dataset.

ID	Application	Flow	ID	Application	Flow
1	allhistory	108	51	sogou	130
2	instagram	4,114	52	baidumap	1,841
3	qqmusic	399	53	baiduhomework	294
4	douyu	649	54	lizhifm	700
5	jxedt	704	55	youku	3,943
6	bilibili	1,390	56	blued	4,786
7	gifmaker	320	57	changba	156
8	TikTok	2,972	58	mtt	619
9	qiyivideo	1,456	59	tuchong	3,507
10	baiduvideo	3,038	60	toutiao	4,324
11	ijinshan	322	61	funshionvideo	308
12	taobao	4,861	62	Lark	2,819
13	TripAdvisor	2,943	63	browser2345	642
14	linkedin	1,524	64	mobiletv	699
15	ctrip	3,366	65	kuwo	213
16	vipshop	6,598	66	kmxsreader	285
17	yangshipin	814	67	letv	237
18	articlevideo	877	68	huya	2,640
19	storm	862	69	ximalaya	540
20	12306MobileTicket	299	70	amap	1,682
21	iReaderFree	432	71	haokan	2,737
22	quark	108	72	duoshan	3,148
23	guotaijunan	2,153	73	tencentnews	863
24	meiyancamera	626	74	baidutieba	1,000
25	qq	102	75	iflytek	117
26	acfundanmaku	385	76	cnvideo	371
27	mtxx	669	77	weibo	2,285
28	airbnb	2,777	78	neteasecloudmusic	3,484
29	huajiao	797	79	mjweather	484
30	lemon	733	80	kugoufanxing	688
31	github	3,813	81	baiduwangpan	815
32	alipay	3,821	82	DCar	1,155
33	Qunar	208	83	zhihu	2,998
34	gogokid	1,234	84	UCMobile	413
35	mmweather	1,013	85	baidusearchbox	3,536
36	Reddit	4,519	86	tencentmap	9,58
37	qqreader	672	87	m2u	98
38	articlenews	1,295	88	superb	909
39	comicsisland	505	89	qqpimsecure	318
40	youthnew	831	90	twitter	2,057
41	pinduoduo	4,738	91	shuqi	415
42	booking	7,127	92	souhutv	342
43	imgo	3,624	93	kuaikan	1,325
44	ugclive	734	94	tujia	521
45	weishi	2,715	95	qqlive	612
46	ele	5,865	96	dragonread	901
47	tudou	1,309	97	weidian	321
48	wuba	491	98	baiduwenku	1,173
49	facebook	2,568	99	meituan	3,052
50	jingdong	3,894	100	anjuke	2,104

each phase, we initialize the class labels list using a random seed to shuffle the list in each repeated experiment. From this shuffled list, classes are randomly selected for both the initial and incremental phases. This method ensures variance in both the category selection for the initial model and the order of class addition during the incremental phases, mitigating potential biases. We randomly select up to $\min(\#Flow, 500)$ flows from each application of all public datasets and $\min(\#Flow, 1,000)$ flows of the newly proposed dataset. These flows are

then divided into the training set, the validation set, and the testing set according to the ratio of 6:1.5:2.5, with a random initialization based on random seeds to prevent potential biases from sample partitioning.

5.2.3. Methods to compare

To obtain a comprehensive understanding of the performance of MISS, we use two existing IL methods of ETC to conduct comparisons,

- **Omt-CNN** (Wu et al., 2022) is an online multimedia incremental framework that uses the sliding window to capture the flow slices for feature extraction. The knowledge distillation and bias correction techniques are applied with old and new samples to overcome the catastrophic forgetting of the standard CNN when new ones appear.
- **CNN-iCarl+** (Bovenzi et al., 2021) is an adapted version based on the classical IL framework, *iCarl* (Rebuffi et al., 2017), for the ETC task, which uses a classification layer with Softmax to replace the nearest mean classifier of *iCarl* and a dynamic output layer expansion fitting the number of new applications.

In an effort to create more detailed comparisons, we introduce two methods of migration as shown in Table 5,

- **Mv Frame Migration.** We utilize the multi-view (Mv) framework MIMETIC (Aceto et al., 2019b), developed for CNN network architectures, to overhaul the feature construction of both the OmtCNN and the CNN-Model, with maintaining the same types of feature input, resulting in **Omt-CNN (Mv)** and **CNN-iCarl+ (Mv)**. For instance, we extract features such as Packet length, Inter arrival time, and Packet direction by the 1d-CNN of CNN-iCarl+, respectively. We then employ the merged layer and associated part of the MIMETIC framework to amalgamate these features before additional processing.
- **IL Frame Migration.** For comparison with SOTA ETC models (without incremental ability), we migrate the *iCarl* framework to the EBSNN (Xiao et al., 2021) and ET-BERT (Lin et al., 2022), resulting in **EBSNN (IL)** and **ET-BERT (IL)**. As suggested by (Bovenzi et al., 2023, 2021), we retain the overall structures (input layer, feature extraction layer, and output layer) and training methods of SOTA models while adding the *iCarl's model parameter passing*, *specific loss function*, and *exemplar selection* components to give them incremental ability. For instance, we use the ET-BERT's model structure as the initial model in the 0th phase, and feed the initial model parameters to the 1th phase by *iCarl's model parameter passing* in order to replace ET-BERT's loss with *iCarl's specific loss function* and select old samples by *iCarl's exemplar selection* after each incremental phase.

5.2.4. Evaluation metrics

The primary goal of ETC is Accuracy, namely, correctly identifying more encrypted traffic flows and avoiding misclassification (Xu et al., 2022). However, considering the long-tail data distribution nature of the dataset, we use an additional evaluation metric, F1-score, which considers both Precision and Recall. Meanwhile, we adopt the macro-average, which averages the performances of each individual application (k applications here),

$$ACC = \frac{Accuracy_1 + \dots + Accuracy_k}{k}$$

$$F1 = 2 * \frac{(\frac{1}{k} \sum_{i=1}^k Pre_i) * (\frac{1}{k} \sum_{i=1}^k Rec_i)}{\frac{1}{k} \sum_{i=1}^k Pre_i + \frac{1}{k} \sum_{i=1}^k Rec_i} \quad (9)$$

where ACC is macro-average Accuracy, $F1$ is macro-average F1-score, Pre is Precision, and Rec is Recall. The mean and standard deviation of metrics are reported by repeating 3 times to ensure the robustness of the results.

Table 5

Summary of the compared methods. Mv Frame means MIMETIC multi-view framework. IL Frame means iCarl incremental learning framework.

Method	Network	Feature Input	Mv Frame Migration	IL Frame Migration
Omt-CNN (Mv)	CNN	Packet length, Inter arrival time, Byte rate	✓	
CNN-iCarl+ (Mv)	CNN	Packet length, Inter arrival time, Packet direction	✓	
EBSNN (IL)	RNN	Header segment, Payload segment		✓
ET-Bert (IL)	Transformer	Payload (with Pre-training)		✓

Table 6

Average incremental Accuracies (avg.ACC)% and F1-score (avg.F1)% on Cross-Platform (iOS) and PURE-TLS datasets.

Method	Dataset Phase	Cross-Platform (iOS)				PURE-TLS			
		UB	N=1	N=10	N=25	UB	N=1	N=10	N=25
Omt-CNN	avg.ACC	85.25 ± 0.31	79.70 ± 0.60	76.90 ± 0.60	74.24 ± 0.71	87.04 ± 0.45	81.22 ± 0.56	75.02 ± 0.55	75.72 ± 0.60
	avg.F1	82.99 ± 0.25	77.25 ± 0.63	75.63 ± 0.63	71.12 ± 0.46	82.80 ± 0.44	76.60 ± 0.63	74.99 ± 0.63	70.85 ± 0.56
Omt-CNN (Mv)	avg.ACC	87.93 ± 0.38	82.37 ± 0.51	79.08 ± 0.33	76.27 ± 0.56	89.57 ± 0.34	83.97 ± 0.40	77.19 ± 0.42	78.75 ± 0.59
	avg.F1	85.62 ± 0.34	80.31 ± 0.57	78.51 ± 0.38	73.92 ± 0.64	85.05 ± 0.29	79.98 ± 0.44	74.99 ± 0.44	73.68 ± 0.63
CNN-iCarl+	avg.ACC	90.27 ± 0.32	85.81 ± 0.49	84.90 ± 0.37	83.21 ± 0.48	84.32 ± 0.58	82.62 ± 0.38	81.98 ± 0.43	77.18 ± 0.57
	avg.F1	84.06 ± 0.28	82.79 ± 0.58	82.74 ± 0.42	80.61 ± 0.49	82.84 ± 0.60	80.45 ± 0.35	81.07 ± 0.72	75.53 ± 0.53
CNN-iCarl+ (Mv)	avg.ACC	92.13 ± 0.36	85.99 ± 0.45	83.12 ± 0.45	79.56 ± 0.40	90.22 ± 0.44	85.78 ± 0.56	83.90 ± 0.35	79.38 ± 0.57
	avg.F1	85.21 ± 0.42	82.67 ± 0.51	79.70 ± 0.38	78.72 ± 0.43	88.87 ± 0.39	84.95 ± 0.60	82.01 ± 0.31	79.61 ± 0.60
EBSNN (IL)	avg.ACC	92.12 ± 0.74	87.22 ± 0.51	86.38 ± 0.74	83.10 ± 0.75	87.59 ± 0.79	82.75 ± 0.44	78.86 ± 0.63	76.06 ± 0.68
	avg.F1	86.71 ± 0.81	83.33 ± 0.49	82.51 ± 0.60	79.73 ± 0.81	86.22 ± 0.85	81.35 ± 0.37	77.96 ± 0.59	75.88 ± 0.79
ET-Bert (IL)	avg.ACC	98.84 ± 0.24	95.60 ± 0.33	95.54 ± 0.12	95.33 ± 0.17	98.68 ± 0.06	96.11 ± 0.29	95.39 ± 0.58	94.61 ± 0.42
	avg.F1	97.20 ± 0.14	94.01 ± 0.37	93.37 ± 0.12	92.92 ± 0.15	97.35 ± 0.27	95.97 ± 0.29	93.48 ± 0.51	94.44 ± 0.37
MISS-CNN	avg.ACC	95.87 ± 0.50	93.89 ± 0.27	92.85 ± 0.32	89.13 ± 0.24	90.40 ± 0.26	86.45 ± 0.38	84.64 ± 0.21	82.15 ± 0.37
	avg.F1	91.06 ± 0.44	89.84 ± 0.32	89.21 ± 0.32	84.68 ± 0.27	87.60 ± 0.27	83.86 ± 0.40	81.26 ± 0.26	79.22 ± 0.39
MISS-TF	avg.ACC	98.12 ± 0.31	97.18 ± 0.32	97.03 ± 0.2	96.71 ± 0.34	98.24 ± 0.25	96.84 ± 0.21	97.29 ± 0.35	96.40 ± 0.23
	avg.F1	97.71 ± 0.25	97.03 ± 0.28	96.56 ± 0.19	96.09 ± 0.36	98.34 ± 0.28	97.06 ± 0.20	97.46 ± 0.37	96.61 ± 0.17

5.2.5. Implementation details

To evaluate the impact of the initial model, we use Text-CNN (Chen, 2015) and Transformer (Vaswani et al., 2017), the advanced models with promising sequence modeling capabilities, as the *multi-view sequence features extraction networks*, respectively. For Text-CNN, we use a set of SGD optimizers with a momentum of 0.9 and a batch size of 256, one with a learning rate of 0.01 to train the ETC model and another with a learning rate of 1×10^{-6} to learn ϵ . For Transformer, we only replace the training model optimizer by BertAdam with a learning rate of 0.05 in the 0th phase. And the incremental phases are set up in the same way as TextCNN. We impose a constraint to ensure that $\epsilon^p + \epsilon^s = 1$, initialized to $[\epsilon^p = 0.5, \epsilon^s = 0.5]$. Empirically, we put S and T in the preliminary experiments as 10 and 10%. These hyperparameter settings perform best on our development set.

5.3. Comparison with IL methods and SOTA models of ETC

We evaluate the classification accuracy and evolution performance of MISS with four datasets, as shown in Table 6 and Table 7, where the average incremental *ACC* (avg.ACC) and *F1* (avg.F1), i.e., the average values of metrics in all phases, are reported as the final evaluation. Note that the phase-wise ACC and F1 in each phase of Cross-Platform (iOS) and *PURE-TLS* datasets can be found in Fig. 8. Two IL methods of ETC, Omt-CNN, CNN-iCarl+, and two SOTA ETC models using the *iCarl* IL framework, EBSNN (IL) and ET-Bert (IL), are used as comparison approaches (details in Section 5.2.3). Note that we produce two variants of the MISS framework, MISS-CNN and MISS-TF, with different feature extraction networks, Text-CNN and Transformer (details in Section 5.2.5). For the purpose of better comparison, we state here two concepts: in each phase, the result of retraining (without any incremental measures) the new model using all the resources, i.e., all the training datasets $D^{0:i}$, is called the **Upper Bound** (UB); correspondingly, the result of using the restricted resources in each phase, i.e., the training data set D^i of the current phase and the exemplar set $\lambda^{0:i-1}$ of the previous phases, is called the **Lower Bound** (LB).

5.3.1. Analysis of different initial models

Under the same IL framework, the UB of MISS-TF is improved by 2.25% and 6.65% compared with MISS-CNN on Cross-Platform (iOS), whose excellent performance is inherited in incremental phases, i.e., the average drop (avg.ACC/F1 of UB – avg.ACC/F1 of N) of MISS-TF is 2.71%↓ and 2.03%↓ less than MISS-CNN. Similarly, the same trend is observed between ET-Bert (IL) and others using the *iCarl* IL framework, i.e., EBSNN (IL) and EBSNN (IL), more pronounced for $N = 25$.

5.3.2. Analysis of different IL frameworks

Under similar Transformer feature extraction networks, MISS-TF, without pre-training, does not perform as well as the pre-training ET-Bert (IL) in the UBs of multiple datasets, but outperforms ET-Bert (IL) in the evolution of all datasets. MISS-TF achieves the best performance on avg.ACC and avg.F1 of Cross-Platform (iOS) as 97.18% and 97.03%, which are only 0.94%↓ and 0.68%↓ drops compared to the UB, 1.58% and 3.02% higher compared to ET-Bert (IL). Under similar CNN feature extraction networks, MISS-CNN achieves the best results compared with different IL frameworks, i.e., Omt-CNN and CNN-iCarl+. MISS-CNN only drops by 1.98%↓ on avg.ACC when N is 1, but Omt-CNN by 5.55%↓ and CNN-iCarl+ by 4.46%↓.

In comparing the two groups that employed the Mv Frame migration methods, the multi-view feature approach is proven to improve the UB performance of the models, providing a solid start for the incremental phase. The UB of Omt-CNN (Mv) outperforms Omt-CNN by 2.68% and 2.63%, respectively, and correspondingly, the incremental phase has also improved by an average of 2.29% and 2.91%. However, although the UB of CNN-iCarl+ (Mv) is improved, the incremental phase of some datasets sees a decrease, potentially due to compatibility issues brought about by the Mv Frame method.

Takeaway. A well-designed IL framework combined with a matching feature extraction architecture can significantly mitigate the degradation of model classification performance. Moreover, establishing a high Upper Bound for the initial model proves to be a crucial factor in its successful evolution.

Table 7

Average incremental Accuracies (avg.ACC)% and F1-score (avg.F1)% on Cross-Platform (Android) and USTC-TFC-2016 datasets.

Method	Dataset Phase	Cross-Platform (Android)				USTC-TFC-2016		
		UB	N=1	N=10	N=25	UB	N=1	N=5
Omt-CNN	avg.ACC	84.63±0.33	80.59±0.44	79.08±0.53	74.29±0.35	86.02±0.46	79.20±0.38	76.17±0.46
	avg.F1	80.49±0.34	74.77±0.52	72.29±0.59	70.08±0.34	86.48±0.38	78.73±0.39	77.53±0.42
Omt-CNN (MV)	avg.ACC	88.19±0.34	83.56±0.53	82.21±0.30	77.60±0.49	89.05±0.52	81.76±0.51	79.84±0.43
	avg.F1	84.12±0.29	77.49±0.59	75.94±0.34	73.96±0.51	89.55±0.59	81.30±0.42	80.62±0.49
CNN-iCarl +	avg.ACC	87.59±0.52	83.68±0.49	81.91±0.50	79.08±0.60	89.83±0.50	85.08±0.55	84.38±0.44
	avg.F1	85.24±0.50	84.12±0.58	81.35±0.56	78.51±0.70	88.06±0.61	83.86±0.47	83.59±0.42
CNN-iCarl + (MV)	avg.ACC	88.86±0.36	84.41±0.36	82.61±0.47	78.28±0.36	91.70±0.45	86.04±0.58	85.84±0.55
	avg.F1	83.83±0.33	82.72±0.34	80.08±0.51	77.14±0.38	89.98±0.53	85.87±0.63	84.65±0.45
EBSNN (IL)	avg.ACC	90.32±0.52	85.17±0.68	84.28±0.34	81.45±0.37	81.41±0.35	72.36±0.74	76.46±0.36
	avg.F1	85.79±0.46	81.98±0.61	81.84±0.41	78.64±0.32	78.93±0.41	69.97±0.86	71.31±0.32
ET-Bert (IL)	avg.ACC	98.74±0.19	94.99±0.30	94.93±0.25	94.06±0.24	99.51±0.22	93.96±0.37	95.71±0.06
	avg.F1	94.28±0.18	92.56±0.27	92.32±0.29	91.38±0.22	99.57±0.20	93.34±0.31	95.55±0.05
MISS-CNN	avg.ACC	90.20±0.25	88.76±0.22	85.75±0.33	80.37±0.21	99.03±0.38	97.57±0.33	97.54±0.25
	avg.F1	88.89±0.23	87.55±0.20	83.23±0.31	77.00±0.19	98.81±0.36	97.76±0.35	97.70±0.28
MISS-TF	avg.ACC	97.77±0.30	97.12±0.20	97.11±0.24	96.52±0.22	99.43±0.33	97.54±0.28	96.38±0.24
	avg.F1	97.02±0.33	95.88±0.18	95.28±0.21	95.23±0.25	99.15±0.36	98.38±0.29	96.97±0.22

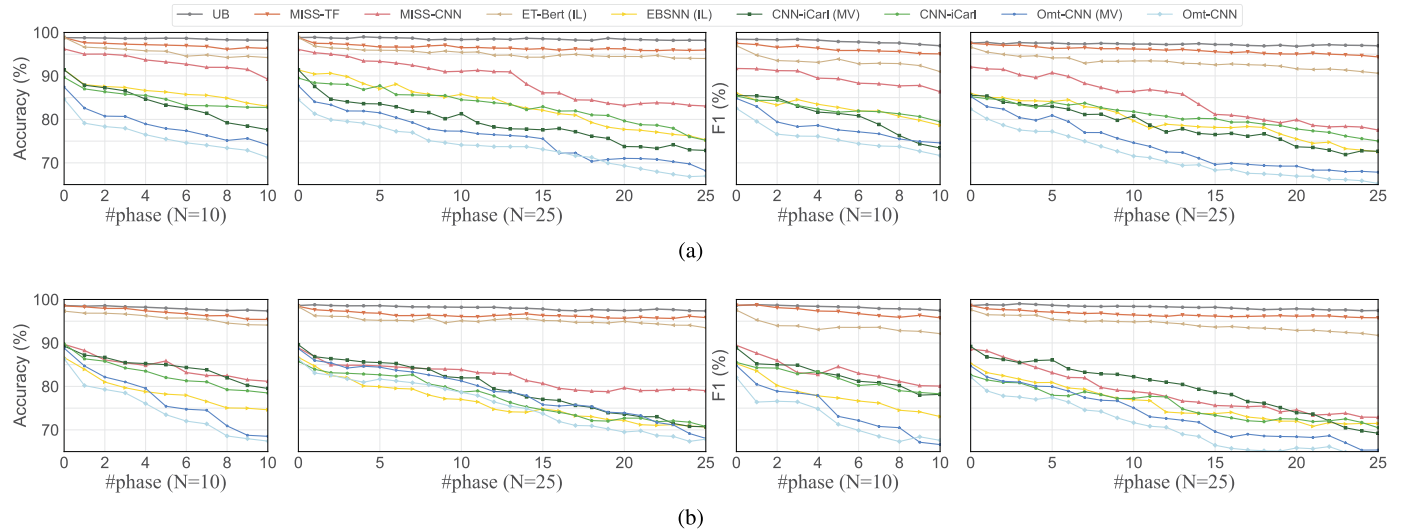


Fig. 8. Phase-wise Accuracies (%) and F1-score (%) of Cross-Platform (iOS) and PURE-TLS datasets. (a) Accuracies and F1-score of Cross-Platform (iOS). In the 0th phase, the old model is trained on 90 applications; the remaining applications are given evenly in the subsequent phases. (b) Accuracies and F1-score of PURE-TLS. In the 0th phase, the initial model is trained on 50 applications; the remaining applications are given evenly in the subsequent phases.

5.4. Incremental evaluations of ETC

We comprehensively explore the incremental effects of representative exemplar selection and design rich incremental experiments to explore the effects of different incremental strategies, focusing on the **Transformer-based MISS-TF** (hereinafter referred to as, **MISS**) and **PURE-TLS** dataset.

5.4.1. Comparison with different exemplar selections

To verify the superiority of our proposed representative exemplar selection, we pick the *iCarl*'s exemplar selection (*iCarl-ES*) and *Random* exemplar selection (*Random-ES*) as comparison schemes to replace the exemplar selection *MISS-ES* of *MISS*, and gradually reduce $\mathcal{T}\%$ from [10%, 5%, 2%], to explore different exemplar selections with different exemplar retention rates. We can see two apparent trends as shown in Fig. 9 and Tables 8-9,

- *MISS-ES* can select more representative exemplars, shows significant superiority over *iCarl-ES* and *Random-ES*, and keep the better classification effect on LB. The increase is most pronounced when

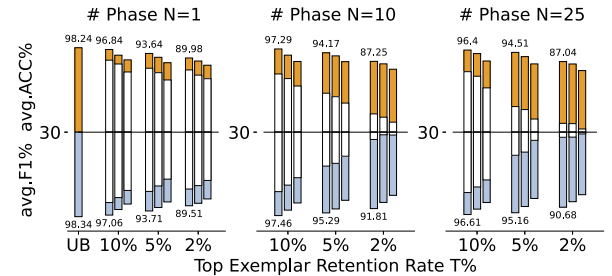


Fig. 9. Schematic of avg.ACC and avg.F1 of different exemplar selections. 30 implies the starting point for evaluation metric values. Each part from left to right is *MISS-ES*, *iCarl-ES*, and *Random-ES*, and the white part is the re-trained Lower Bound (LB) classification effect.

$\mathcal{T}\% = 2\%$, as shown in Table 8. Under extreme LB conditions, *MISS-ES* significantly improves avg.F1 scores over *iCarl* and *Random* exemplar selection, underscoring the effectiveness of our exemplar selection strategy.

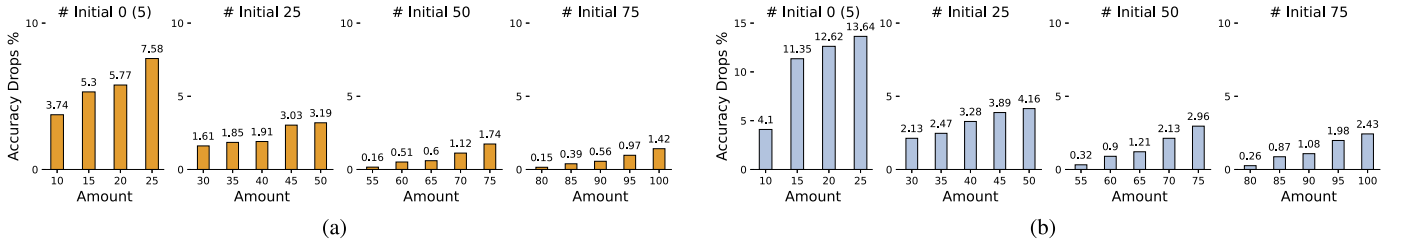


Fig. 10. The accuracy drops after each incremental phase under the different initial application amounts. (a) The accuracy drops in all applications when new applications are added. (b) The accuracy drops in initial applications when new applications are added.

Table 8

Compared with iCarl and Random exemplar selections on the avg.F1 when $\mathcal{T}\% = 2\%$. $\downarrow$ means the decline compared with MISS.

Method	N = 1	N = 10	N = 25
MISS-ES	89.51	91.81	90.68
iCarl-ES	$\downarrow 2.02(87.49)$	$\downarrow 5.02(86.79)$	$\downarrow 4.45(86.23)$
Random-ES	$\downarrow 5.65(83.86)$	$\downarrow 10.73(81.08)$	$\downarrow 10.11(80.57)$
MISS-ES-LB	75.96	35.92	34.14
iCarl-ES-LB	$\downarrow 2.75(73.21)$	$\downarrow 3.80(32.12)$	$\downarrow 0.12(34.02)$
Random-ES-LB	$\downarrow 6.94(69.02)$	$\downarrow 3.59(32.33)$	$\downarrow 2.71(31.42)$

- A higher exemplar retention rate $\mathcal{T}\%$ improves the evolution effect but increases resource consumption. Total.RT represents the cumulative runtime across all $(N+1)$ phases, avg.RT denotes the average runtime per phase, Storage refers to the actual traffic storage resources utilized in the final $(N+1)^{\text{th}}$ phase, as Table 9 shows. This also aligns with our cognition: a higher exemplar retention rate translates to increased runtime and storage resources.

Takeaway. Reasonable exemplar selection can improve the evolution effect. $\mathcal{T}\%$ is the art of balancing evolution performance with computational resource consumption. To pursue a faster evolution is achieved by the reasonable exemplar selection according to a low $\mathcal{T}\%$, which could not be neglected to ensure that the absolute amount of exemplars should remain in an appropriate interval.

5.4.2. Comparison of different initial application amounts

We now discuss how performance is affected by different initial application amounts in the 0^{th} phase. In Fig. 10(a), we compare the performance of the initial model trained on the amount from [0, 25, 50, 75], and execute up to 5 incremental phases, each adding 5 new applications. To complement the analysis, we give the classification ability of the initial applications in the 0^{th} phase (when the amount is 0, the 1^{st} phase is analyzed) after new applications are added, as shown in Fig. 10(b). We can observe an overlapping trend: under the incremental application amount is fixed, the fewer initial applications, the more significantly decreases the accuracy of all applications and old applications after each incremental phase, especially when the initial applications are less than half the total applications. Meanwhile, the accuracy of initial applications can be severely detrimental ($\downarrow 13.64\%$) at the final incremental phase when too few initial applications are available.

Takeaway. Fewer initial applications in the 0^{th} phase, the more negative impact of incremental phases on accuracy. For an evolving ETC model, a full training should be performed after several incremental phases, resetting the 0^{th} phase.

5.4.3. IL strategies for homogeneous applications

We observe homogeneity in the PURE-TLS dataset: the applications from the same families and types, e.g., Taobao and Alipay are both from the Alibaba family, Facebook and Instagram are both social applications. We set the initial application amount as 50, with $\mathcal{T}\% = 10\%$, and set up three sets of experiments:

Table 9

Runtime (h) and Storage (GB) of $N+1$ phases under $\mathcal{T}\% = [10\%, 5\%, 2\%]$.

Phase	Metrics	UB	$\mathcal{T}\% = 10\%$	$\mathcal{T}\% = 5\%$	$\mathcal{T}\% = 2\%$
N = 1	total.RT (h)	3.23	2.41	2.27	2.07
	avg.RT (h)	1.62	1.21	1.14	1.04
	Storage (GB)	41.91	21.65	20.29	19.43
N = 10	total.RT (h)	15.37	3.72	3.23	2.63
	avg.RT (h)	1.40	0.34	0.29	0.24
	Storage (GB)	41.91	6.65	4.47	3.05
N = 25	total.RT (h)	38.66	5.53	4.47	3.15
	avg.RT (h)	1.49	0.21	0.17	0.12
	Storage (GB)	41.91	5.72	3.43	1.82

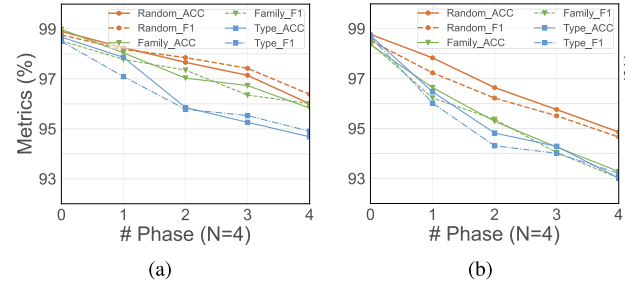


Fig. 11. Phase-wise metrics on IL strategies of homogeneous applications. (a) The metrics of MISS. (b) The metrics of ET-Bert (IL).

- Randomly select 40 applications in 4 phases.
- Select 10 applications from each Alibaba, Baidu, ByteDance, and Tencent family.
- Select 10 applications from each Shopping, Social, Reading, and Video type.

We make a comparison with ET-Bert (IL) for further analysis, and the results are shown in Fig. 11. The IL Strategies of homogeneous applications reduce the gain of the IL framework, but MISS possesses certain robustness, especially for the same family. Fig. 12 visualizes the embedding feature spaces of MISS and ET-Bert (IL) using t-SNE (Van der Maaten and Hinton, 2008), which intuitively demonstrates that the embeddings of flows within the same family generated by MISS are more compact and separable (especially inside the circle box). We argue that MISS deals with the difficulty of classifying homogenous applications from the same families and types by considering features from different views. For homogeneity problems specific to ETC, the multi-view sequence features perform better. We provide the ablation experiments for different view sequence features to enable a better demonstration, as shown in Fig. 13. Different view sequence features perform differently in incremental phases on the homogenous applications, e.g., the homogenous types are better in the content type view than other views.

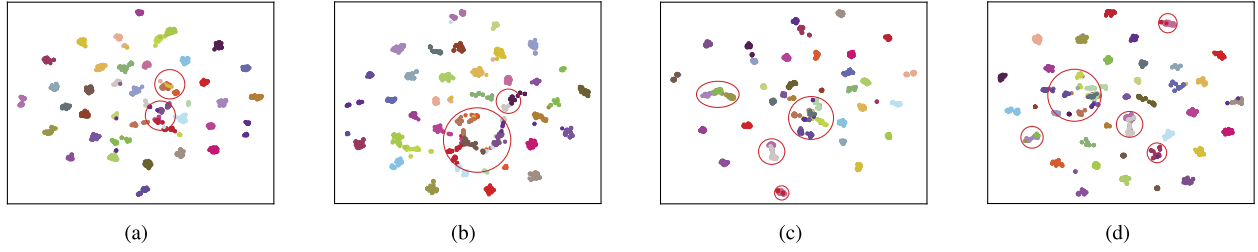


Fig. 12. Visualizations of MISS and ET-Bert (IL) using t-SNE in the final phase respectively. (a) Visualizations of MISS with homogenous family. (b) Visualizations of ET-Bert (IL) with homogenous family. (c) Visualizations of MISS with homogenous type. (d) Visualizations of ET-Bert (IL) with homogenous type.

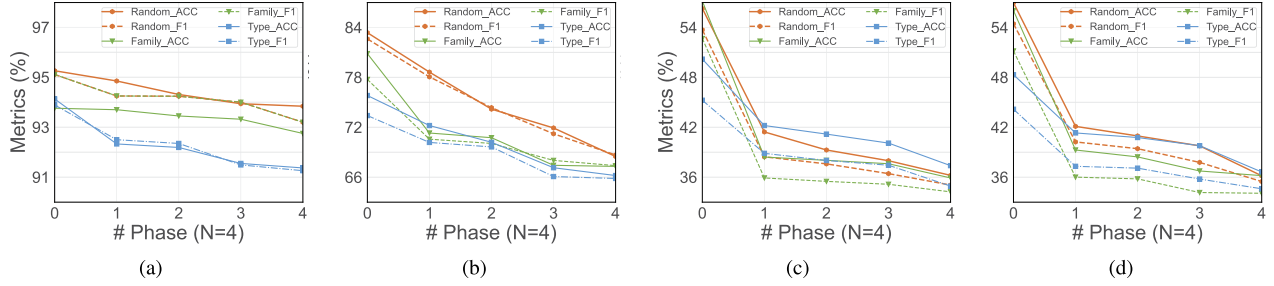


Fig. 13. Phase-wise metrics on homogeneous applications for different view sequence features. (a) The payload byte view. (b) The packet length view. (c) The TLS content type view. (d) The arrival time view.

Takeaway. Once encountering a severe group of homogeneous applications in IL, it will reduce the performance after evolution. Therefore, we suggest that the variability of the added applications should be guaranteed, which could be artificially divided with prior information.

5.4.4. Comparison of CV IL framework migration

To delve deeper into the performance of MISS in the context of encrypted traffic tasks, we embark on an exploration of migrating IL frameworks from the CV field, under the guidance of (Bovenzi et al., 2023, 2021), to carry out further comparison.

- **iCarl:** As delineated, *iCarl* (Rebuffi et al., 2017) amalgamates a specific loss. It introduced the exemplar selection, underscoring the efficacy of the nearest-mean-of-exemplars classification in IL scenarios.
- **DER:** Dynamical Expandable Representations (DER) (Yan et al., 2021) appends a fresh feature extractor at each phase, updating it while keeping previous ones. For the i^{th} phase, outputs from all $i + 1$ extractors are combined for classification.

We retain the feature extraction capabilities of the 0^{th} phase model and migrate *iCarl* and DER IL frameworks, respectively, to form the *iCarl-M²* and DER-M² variants. These variants are benchmarked against MISS on 5 traffic tasks, with a focus on $N = 1$ incremental phase, as shown in Table 10:

- **Task#1:** ETC of iOS Applications using the Cross-Platform (iOS) dataset for this investigation.
- **Task#2:** For ETC on Android Applications, both the Cross-Platform (Android) and MIRAGE-2019 (Aceto et al., 2019a) datasets are considered. MIRAGE-2019 analyzes traffic from 40 applications on 3 Android devices, with its public subset covering 20 applications on 2 devices. For the sake of brevity, we focus on traffic from the Xiaomi Mi5 using 10 as the initial number of classes.
- **Task#3:** For ETC on Malicious Applications, we use the USTC-TFC-2016 dataset.
- **Task#4:** We utilize the PURE-TLS dataset for Pure-encrypted Traffic Classification task.

- **Task#5:** The ISCX-17 dataset (Draper-Gil et al., 2016) is selected for the VPN-encrypted Traffic Classification task. It encompasses seven traffic types related to both VPN and non-VPN communications, originating from various applications. This results in 17 distinct classes, and we use 8 as the initial number of classes for our analysis.

On Task#1, MISS achieves an avg.ACC of 97.18% and an avg.F1 of 97.03%, outstripping *iCarl-M²* by margins of 1.76% and 3.59%, and DER-M² by 0.92% and 1.71%, respectively. On Task#2, for the Cross-Platform (Android), MISS surpasses *iCarl-M²* by 2.76% in avg.ACC and 5.46% in avg.F1, and outpaces DER-M² by 1.55% and 3.53%, respectively. For the MIRAGE-2019 dataset, MISS exceeds *iCarl-M²* by 4.78% in avg.ACC and 6.17% in avg.F1, and outclasses DER-M² by 2.10% and 3.29% in the same metrics. On Task#3, the robustness of MISS is evident, overshadowing *iCarl-M²* by a significant 5.09% in avg.ACC and 7.09% in avg.F1. Task#4 and Task#5 further highlight MISS's versatility. In the PURE-TLS dataset, MISS leads *iCarl-M²* by 1.42% in avg.ACC and 2.10% in avg.F1, and surpasses DER-M² by 0.97% and 1.64%. Even within the ISCX-17 dataset, MISS maintains its superiority, outperforming *iCarl-M²* and DER-M² by 3.02% and 0.80% in avg.ACC, respectively.

Takeaway. MISS, tailored for encrypted traffic, consistently outshines CV-focused IL frameworks. Its robust performance across varied datasets underscores its potential as a benchmark in encrypted traffic tasks.

5.4.5. IL strategies for unknown applications

We envision the scenario in which the initial model discovers unknown applications rather than acquiring manually labeled applications for evolution, discovering unknown, clustering unknown, and using clusters as new applications for evolution. The IL Problem of ETC is transformed into feeding the i^{th} phase test set $D_{\text{test}}^{0:i}$ and the $(i + 1)^{\text{th}}$ phase D^{i+1} to the i^{th} phase model $\mathcal{M}^i$ during the testing process, discovering the samples of D^{i+1} and clustering them. It is not necessary to evolve on all samples of D^{i+1} , and the correctly obtained from the discovering and clustering process are enough (Leo and Kalita, 2021). We use OpenMax instead of SoftMax to equip MISS with the ability to reject unknown applications. OpenMax is training independently as described

Table 10Comparison of avg.ACC and avg.F1 scores for iCarl-M², DER-M², and MISS across 5 traffic classification tasks.

Method	N = 1	Task#1	Task#2	Task#3	Task#4	Task#5	
	Metrics	Cross-Platform (iOS)	Cross-Platform (Android)	MIRAGE-2019	USTC-TFC-2016	PURE-TLS	ISCX-17
UB	avg.ACC	98.12	97.77	82.10	99.43	98.24	88.35
	avg.F1	97.71	97.02	78.94	99.15	98.34	84.52
iCarl-M ²	avg.ACC	95.42	94.42	74.86	92.45	95.53	82.46
	avg.F1	93.44	90.42	71.65	91.29	94.96	80.29
DER-M ²	avg.ACC	96.26	95.57	77.54	96.85	95.87	84.68
	F1	95.32	92.35	74.53	95.28	95.42	82.01
MISS	avg.ACC	97.18	97.12	79.64	97.54	96.84	85.48
	avg.F1	97.03	95.88	77.82	98.38	97.06	81.81

Table 11

The average metrics for the different counts of phase. The values in parentheses correspond to the results from Table 6.

Phases	avg.ACC _{os}	avg.ACC↓	avg.F1 _{os}	avg.F1↓
N = 1	85.76	↓11.08(96.84)	84.88	↓12.18(97.06)
N = 10	88.86	↓8.43(97.29)	87.43	↓10.03(97.46)
N = 25	91.29	↓5.11(96.40)	92.14	↓4.47(96.61)

in (Bendale and Boulton, 2016), which uses a confidence value to identify open set data. OpenMax revises SoftMax activation vectors adding a special “synthetic” unknown node (by using weighting induced by Weibull). We follow existing work (Lyu et al., 2023; Oveis et al., 2022) and set the Weibull distribution set to 20. The clustering algorithm is Modified PCKMeans, which uses 3-tuples (IP address, port, TLS certificate) as pairwise constraints to guide the clustering, and we respect the parameter settings in (Zhang et al., 2019). Following (Bendale and Boulton, 2015; Ge et al., 2017), we use the Open-Set metrics to measure the evaluation performance (ACC_{os}, F1_{os}), e.g., ACC_{os} is defined as,

$$ACC_{os} = \frac{1}{D_{test}^{0:i}} \sum_{j=1}^{D_{test}^{0:i}} [\mathcal{M}^i(\mathbf{x}_j) = y_j] + \frac{1}{D^{i+1}} \sum_{j=1}^{D^{i+1}} [\mathcal{M}^i(\mathbf{x}_j) = \text{UNK}] \quad (10)$$

Results are shown in Table 11. Compared to the manually labeled in Table 6, the classification effect is severely reduced when the number of unknown applications is similar to the initial applications (when $N = 1$). It also gives relatively better results when the added applications in each phase are much smaller than the initial applications (when $N = 25$) because this limits the error in the non-incremental part.

Takeaway. The non-incremental component mainly influences the evolution effect of unknown applications, which is an inevitable negative effect. Errors in the discovering and clustering processes increase and propagate when more unknown applications are added simultaneously in one phase.

6. Discussion

6.1. Implication

Our observations in this paper can provide implications for exploration. We find that an ETC model with a well-designed IL framework could keep evolving well when new applications emerge, significantly reducing resource consumption while maintaining excellent performance. Firstly, in our observations, a high Upper Bound of the initial model is crucial for its evolution. We suggest building an excellent initial model as much as possible at the feature design level or model structure level. Secondly, the exemplar retention rate $\mathcal{T}\%$ is the art of balancing evolution performance with computational resource consumption. A lower $\mathcal{T}\%$ can be appropriately selected to reduce the evolution runtime significantly. Thirdly, we find that the fewer initial applications in the 0th phase, the higher the loss of accuracy due to evolution. We suggest a full training of all samples should be performed

after several evolution phases to reset the 0th phase. Finally, the problem of homogeneous applications is not negligible. We suggest that the variability of the added applications should be guaranteed, which could be artificially divided with prior information.

6.2. Limitations

Experiments show that our MISS framework has excellent performance, but there are some limitations.

Firstly, from the dataset perspective, the PURE-TLS dataset we constructed contains encrypted traffic data from 100 applications. However, some applications have less data, and the dataset as a whole exhibits an unbalanced phenomenon. This is because many applications are not pure TLS traffic at runtime, and we removed all non-TLS traffic during preprocessing. This treatment during data collection may result in the loss of some important information, particularly for applications that generate a substantial amount of non-TLS traffic during operation. Therefore, although this treatment helps us focus on analyzing TLS traffic, we recognize that it may limit the generalizability of our dataset. In the future, we will continuously update the PURE-TLS dataset to increase the number of application types and the scale of traffic flows.

Secondly, regarding our experiments, selecting a maximum of 500 (or 1000) flows might diminish the variability among the classes. In our future endeavors, we aim to adopt proportional undersampling to preserve class differences and more accurately capture their inherent variability. Further, classifying applications from the same cloud provider, given their overlapping APIs and services, needs deeper exploration. Such overlaps challenge ETC models, leading to reduced accuracy and F1-score for MISS in these contexts.

Finally, MISS is a lightweight ETC model with excellent performance in multiple ETC tasks, but there is still potential for enhancement. For exemplar, pre-training technology has achieved promising results in many research fields, which can help MISS further improve the processing capability for ETC tasks, especially the detection capability for unknown applications, and to address the performance decreasing on non-exactly labeled increments.

7. Conclusion

In this paper, we proposed an IL framework named MISS allowing the ETC models to keep evolving when new applications emerge. MISS is more applicable to sequence features of ETC than the existing IL frameworks and migration work. We conclude a comprehensive experimental analysis and discussion to evaluate the feasibility and robustness nature of MISS, which improve 11.37%↑ and 1.58%↑ compared to the existing IL methods of ETC and the SOTA ETC models using classic IL framework. Furthermore, we comprehensively perform incremental experiments to evaluate the evolution ability of MISS, by comparison with different exemplar selections, different initial application amounts, homogeneous applications increments, and unknown applications increments, and obtain a series of valuable findings. We believe our research work can positively contribute to the vision of on-

line evolution in traffic classification models and automation in traffic analysis systems.

CRedit authorship contribution statement

Xiang Li: Conceptualization, Data curation, Formal analysis, Methodology, Software, Validation, Visualization, Writing – original draft, Writing – review & editing. **Jiang Xie:** Conceptualization, Formal analysis, Investigation, Methodology, Resources, Writing – review & editing. **Qige Song:** Conceptualization, Investigation, Methodology, Writing – review & editing. **Yafei Sang:** Data curation, Project administration, Supervision, Writing – original draft. **Yongzheng Zhang:** Data curation, Project administration, Resources, Supervision. **Shuhao Li:** Funding acquisition, Resources, Supervision. **Tianning Zang:** Conceptualization, Methodology, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

This work was supported by the National Key Research and Development Program of China (Grant No. 2018YFB0804702), and the National Natural Science Foundation of China (Grant No. U1736218).

References

- Aceto, G., Ciunzo, D., Montieri, A., Persico, V., Pescapé, A., 2019a. Mirage: mobile-app traffic capture and ground-truth creation. In: 2019 4th International Conference on Computing, Communications and Security (ICCCS). IEEE, pp. 1–8.
- Aceto, G., Ciunzo, D., Montieri, A., Pescapé, A., 2017. Traffic classification of mobile apps through multi-classification. In: GLOBECOM 2017-2017 IEEE Global Communications Conference. IEEE, pp. 1–6.
- Aceto, G., Ciunzo, D., Montieri, A., Pescapé, A., 2019b. Mimetic: mobile encrypted traffic classification using multimodal deep learning. *Comput. Netw.* 165, 106944.
- Bendale, A., Boulton, T., 2015. Towards open world recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 1893–1902.
- Bendale, A., Boulton, T.E., 2016. Towards open set deep networks. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 1563–1572.
- Bovenzi, G., Nascita, A., Yang, L., Finamore, A., Aceto, G., Ciunzo, D., Pescapé, A., Rossi, D., 2023. Benchmarking class incremental learning in deep learning traffic classification. *IEEE Trans. Netw. Serv. Manag.*
- Bovenzi, G., Yang, L., Finamore, A., Aceto, G., Ciunzo, D., Pescapé, A., Rossi, D., 2021. A first look at class incremental learning in deep learning mobile traffic classification. *ArXiv preprint arXiv:2107.04464*.
- Bujlow, T., Carela-Español, V., Barlet-Ros, P., 2015. Independent comparison of popular dpi tools for traffic classification. *Comput. Netw.* 76, 75–89.
- Castro, F.M., Marín-Jiménez, M.J., Guil, N., Schmid, C., Alahari, K., 2018. End-to-end incremental learning. In: Proceedings of the European Conference on Computer Vision (ECCV), pp. 233–248.
- Chang, W.L., Sun, H.M., Wu, W., 2016. An android behavior-based malware detection method using machine learning. In: 2016 IEEE International Conference on Signal Processing, Communications and Computing (ICSPCC). IEEE, pp. 1–4.
- Chen, Y., 2015. Convolutional neural network for sentence classification. Master's thesis. University of Waterloo.
- Chen, Y., Zang, T., Zhang, Y., Zhou, Y., Ouyang, L., Yang, P., 2021. Incremental learning for mobile encrypted traffic classification. In: ICC 2021-IEEE International Conference on Communications. IEEE, pp. 1–6.
- Douillard, A., Cord, M., Ollion, C., Robert, T., Valle, E., 2020. Podnet: pooled outputs distillation for small-tasks incremental learning. In: Computer Vision-ECCV 2020: 16th European Conference, Proceedings, Part XX 16. Glasgow, UK, August 23–28, 2020. Springer, pp. 86–102.
- Draper-Gil, G., Lashkari, A.H., Mamun, M.S.I., Ghorbani, A.A., 2016. Characterization of encrypted and vpn traffic using time-related. In: Proceedings of the 2nd International Conference on Information Systems Security and Privacy (ICISSP), pp. 407–414.
- van Ede, T., Bortolameotti, R., Continella, A., Ren, J., Dubois, D.J., Lindorfer, M., Choffnes, D., van Steen, M., Peter, A., 2020. Flowprint: semi-supervised mobile-app fingerprinting on encrypted network traffic. In: Network and Distributed System Security Symposium (NDSS).
- Finsterbusch, M., Richter, C., Rocha, E., Muller, J.A., Hanssger, K., 2013. A survey of payload-based traffic classification approaches. *IEEE Commun. Surv. Tutor.* 16, 1135–1156.
- French, R.M., 1999. Catastrophic forgetting in connectionist networks. *Trends Cogn. Sci.* 3, 128–135.
- Fu, Y., Xiong, H., Lu, X., Yang, J., Chen, C., 2016. Service usage classification with encrypted internet traffic in mobile messaging apps. *IEEE Trans. Mob. Comput.* 15, 2851–2864.
- Ge, Z., Demyanov, S., Chen, Z., Garnavi, R., 2017. Generative openmax for multi-class open set classification. *ArXiv preprint arXiv:1707.07418*.
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., Bengio, Y., 2014. Generative adversarial nets. *Adv. Neural Inf. Process. Syst.* 27.
- Google, 2022. *Https encryption on the web*. <https://transparencyreport.google.com/https/overview?hl=en>.
- Group, I.S.R., 2020. 2020 annual report. <https://letsencrypt.org/>.
- Hendrycks, D., Gimpel, K., 2016. Gaussian error linear units (gelus). *ArXiv preprint arXiv:1606.08415*.
- Hinton, G., Vinyals, O., Dean, J., 2015. Distilling the knowledge in a neural network. *ArXiv preprint arXiv:1503.02531*.
- Hou, S., Pan, X., Loy, C.C., Wang, Z., Lin, D., 2019. Learning a unified classifier incrementally via rebalancing. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 831–839.
- Kang, M., Park, J., Han, B., 2022. Class-incremental learning by knowledge distillation with adaptive feature consolidation. *ArXiv e-prints arXiv:2204.00895*.
- Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A.A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., et al., 2017. Overcoming catastrophic forgetting in neural networks. *Proc. Natl. Acad. Sci.* 114, 3521–3526.
- Korczynski, M., Duda, A., 2014. Markov chain fingerprinting to classify encrypted traffic. In: Infocom. IEEE.
- Lee, I., Roh, H., Lee, W., 2020. Encrypted malware traffic detection using incremental learning. In: IEEE INFOCOM 2020-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS). IEEE, pp. 1348–1349.
- Leo, J., Kalita, J., 2021. Incremental deep neural network learning using classification confidence thresholding. *IEEE Trans. Neural Netw. Learn. Syst.*
- Li, J., Xue, D., Wu, W., Wang, J., 2020. Incremental learning for malware classification in small datasets. *Secur. Commun. Netw.* 2020, 1–12.
- Lin, X., Xiong, G., Gou, G., Li, Z., Shi, J., Yu, J., 2022. Et-bert: a contextualized datagram representation with pre-training transformers for encrypted traffic classification. *ArXiv preprint arXiv:2202.06335*.
- Liu, C., He, L., Xiong, G., Cao, Z., Li, Z., 2019. Fs-net: a flow sequence network for encrypted traffic classification. In: IEEE INFOCOM 2019-IEEE Conference on Computer Communications. IEEE, pp. 1171–1179.
- Liu, H., Wang, Z., Wang, Y., 2012. Semi-supervised encrypted traffic classification using composite features set. *J. Netw.* 7, 1195.
- Liu, J., Fu, Y., Ming, J., Ren, Y., Sun, L., Xiong, H., 2017. Effective and real-time in-app activity analysis in encrypted internet traffic streams. In: Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 335–344.
- Liu, Y., Schiele, B., Sun, Q., 2021. Adaptive aggregation networks for class-incremental learning. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 2544–2553.
- Liu, Y., Su, Y., Liu, A.A., Schiele, B., Sun, Q., 2020. Mnemonics training: multi-class incremental learning without forgetting. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 12245–12254.
- Lotfollahi, M., Jafari Siavoshani, M., Shirali Hossein Zade, R., Saberian, M., 2020. Deep packet: a novel approach for encrypted traffic classification using deep learning. *Soft Comput.* 24, 1999–2012.
- Lyu, Z., Gutierrez, N.B., Beksi, W.J., 2023. Metamax: Improved open-set deep neural networks via Weibull calibration. In: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision, pp. 439–443.
- Van der Maaten, L., Hinton, G., 2008. Visualizing data using t-sne. *J. Mach. Learn. Res.* 9.
- Millar, S., McLaughlin, N., del Rincon, J.M., Miller, P., 2021. Multi-view deep learning for zero-day android malware detection. *J. Inf. Sec. Appl.* 58, 102718.
- Ostapenko, O., Puscas, M., Klein, T., Jähnichen, P., Nabi, M., 2019. Learning to remember: a synaptic plasticity driven framework for continual learning. *ArXiv preprint arXiv:1904.03137*.
- Oveis, A.H., Giusti, E., Ghio, S., Martorella, M., 2022. Open set recognition in sar images using the openmax approach: challenges and extension to boost the accuracy and robustness. In: EUSAR 2022; 14th European Conference on Synthetic Aperture Radar, VDE, pp. 1–4.
- Rebuffi, S.A., Kolesnikov, A., Sperl, G., Lampert, C.H., 2017. icarl: incremental classifier and representation learning. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 2001–2010.

- Ren, J., Dubois, D., Choffnes, D., 2019. An International View of Privacy Risks for Mobile Apps.
- Rescorla, E., Modadugu, N., 2012. Datagram Transport Layer Security Version 1.2.
- Rezaei, S., Liu, X., 2019. Deep learning for encrypted traffic classification: an overview. *IEEE Commun. Mag.* 57, 76–81.
- Rusu, A.A., Rabinowitz, N.C., Desjardins, G., Soyer, H., Kirkpatrick, J., Kavukcuoglu, K., Pascanu, R., Hadsell, R., 2016. Progressive neural networks. *ArXiv preprint arXiv: 1606.04671*.
- Shen, M., Wei, M., Zhu, L., Wang, M., 2017. Classification of encrypted traffic with second-order Markov chains and application attribute bigrams. *IEEE Trans. Inf. Forensics Secur.* 12, 1830–1843.
- Sherry, J., Lan, C., Popa, R.A., Ratnasamy, S.P., 2015. Blindbox: deep packet inspection over encrypted traffic. In: *Computer Communication Review: A Quarterly Publication of the Special Interest Group on Data Communication*.
- Sun, G., Li, S., Chen, T., Su, Y., Lang, F., 2017. Traffic classification based on incremental learning method. In: *International Conference on Advanced Hybrid Information Processing*. Springer, pp. 341–348.
- Sun, Q., Liu, Y., Chua, T.S., Schiele, B., 2019. Meta-transfer learning for few-shot learning. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 403–412.
- Taylor, V.F., Spolaor, R., Conti, M., Martinovic, I., 2017. Robust smartphone app identification via encrypted network traffic analysis. *IEEE Trans. Inf. Forensics Secur.* 13, 63–78.
- Tolstikhin, I.O., Houlsby, N., Kolesnikov, A., Beyer, L., Zhai, X., Unterthiner, T., Yung, J., Steiner, A., Keysers, D., Uszkoreit, J., et al., 2021. Mlp-mixer: an all-mlp architecture for vision. *Adv. Neural Inf. Process. Syst.* 34.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I., 2017. Attention is all you need. *Adv. Neural Inf. Process. Syst.* 30.
- Velan, P., Čermák, M., Čeleda, P., Drašar, M., 2015. A survey of methods for encrypted traffic classification and analysis. *Int. J. Netw. Manag.* 25, 355–374.
- Wang, P., Chen, X., Ye, F., Sun, Z., 2019. A survey of techniques for mobile service encrypted traffic classification using deep learning. *IEEE Access* 7, 54024–54033.
- Wang, W., Zhu, M., Wang, J., Zeng, X., Yang, Z., 2017a. End-to-end encrypted traffic classification with one-dimensional convolution neural networks. In: *2017 IEEE International Conference on Intelligence and Security Informatics (ISI)*. IEEE, pp. 43–48.
- Wang, W., Zhu, M., Zeng, X., Ye, X., Sheng, Y., 2017b. Malware traffic classification using convolutional neural network for representation learning. In: *2017 International Conference on Information Networking (ICOIN)*. IEEE, pp. 712–717.
- Wang, X., Chen, S., Su, J., 2020. Automatic mobile app identification from encrypted traffic with hybrid neural networks. *IEEE Access* 8, 182065–182077.
- Wang, Y., Xiang, Y., Zhang, J., Zhou, W., Wei, G., Yang, L.T., 2013. Internet traffic classification using constrained clustering. *IEEE Trans. Parallel Distrib. Syst.* 25, 2932–2943.
- Welling, M., 2009. Herding dynamical weights to learn. In: *Proceedings of the 26th Annual International Conference on Machine Learning*, pp. 1121–1128.
- Wu, Z., Dong, Y.n., Qiu, X., Jin, J., 2022. Online multimedia traffic classification from the qos perspective using deep learning. *Comput. Netw.* 108716.
- Xiao, X., Xiao, W., Li, R., Luo, X., Zheng, H.T., Xia, S.T., 2021. Ebsnn: extended byte segment neural network for network traffic classification. *IEEE Trans. Dependable Secure Comput.*
- Xu, J., Li, W., Liu, X., Zhang, D., Liu, J., Han, J., 2020a. Deep embedded complementary and interactive information for multi-view classification. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 6494–6501.
- Xu, S.J., Geng, G.G., Jin, X.B., Liu, D.J., Weng, J., 2022. Seeing traffic paths: encrypted traffic classification with path signature features. *IEEE Trans. Inf. Forensics Secur.* 17, 2166–2181.
- Xu, Y., Zhang, Y., Guo, W., Guo, H., Tang, R., Coates, M., 2020b. Graphsail: graph structure aware incremental learning for recommender systems. In: *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, pp. 2861–2868.
- Yan, S., Xie, J., He, X., 2021. Der: dynamically expandable representation for class incremental learning. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 3014–3023.
- Zbontar, J., Jing, L., Misra, I., LeCun, Y., Deny, S., 2021. Barlow twins: self-supervised learning via redundancy reduction. In: *International Conference on Machine Learning*. PMLR, pp. 12310–12320.
- Zhang, C., Song, N., Lin, G., Zheng, Y., Pan, P., Xu, Y., 2021. Few-shot incremental learning with continually evolved classifiers. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 12455–12464.
- Zhang, Y., Zhao, S., Sang, Y., 2019. Towards unknown traffic identification using deep auto-encoder and constrained clustering. In: *International Conference on Computational Science*. Springer, pp. 309–322.
- Zhao, J., Jing, X., Yan, Z., Pedrycz, W., 2021. Network traffic classification for data fusion: a survey. *Inf. Fusion* 72, 22–47.
- Zheng, W., Gou, C., Yan, L., Mo, S., 2020. Learning to classify: a flow-based relation network for encrypted traffic classification. In: *Proceedings of the Web Conference 2020*, pp. 13–22.
- Zhu, K., Cao, Y., Zhai, W., Cheng, J., Zha, Z.J., 2021. Self-promoted prototype refinement for few-shot class-incremental learning. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 6801–6810.
- Zhu, M.y., Chen, Z., Chen, K.f., Lv, N., Zhong, Y., 2022. Attention-based federated incremental learning for traffic classification in the internet of things. *Comput. Commun.* 185, 168–175.

Xiang Li received the B.Eng. degree from Northeast Agricultural University, China, in 2019. He is currently pursuing a Ph.D. degree with the School of Cyber Security, University of Chinese Academy of Sciences, China, and the Institute of Information Engineering, Chinese Academy of Sciences, China. His research interests include mobile security and traffic analysis.

Jiang Xie received the B.Eng. degree from Nankai University, China, in 2018. He is currently a graduate student studying for Ph.D. in the Institute of Information Engineering, Chinese Academy of Sciences, China. His research interests include network security and artificial intelligence.

Qige Song received the B.Eng. from Beijing Jiaotong University, China, in 2017, and the M.Eng. in the Institute of Information Engineering, Chinese Academy of Sciences, China, in 2020. She is currently pursuing the Ph.D. degree with the Institute of Information Engineering, Chinese Academy of Science, China. Her current research interests include IoT security and artificial intelligence.

Yafei Sang received the B.Eng. degree from Shanxi University, China, in 2013, and the Ph.D. degree from the Institute of Information Engineering, Chinese Academy of Sciences, China, in 2018. He is currently an Associate Professor with the Institute of Information Engineering, Chinese Academy of Sciences, China. His research interests include encrypted traffic measurement and analysis, threat detection, application identification, and fingerprint extraction.

Yongzheng Zhang received the B.S. and Ph.D. degrees from the Harbin Institute of Technology, China, in 2001 and 2006, respectively. He was a professor and doctoral supervisor at the Institute of Information Engineering, Chinese Academy of Sciences, and now works in the China Assets Cybersecurity Technology CO., Ltd., China. His research interests include network security, particularly cyberspace security situational awareness. He was honored with the First Prize of the Chinese National Award for Science and Technology Progress in 2011.

Shuhao Li received the B.Eng. and M.Eng. degrees from the Harbin Institute of Technology, China, in 2006 and 2008, respectively, and the Ph.D. degree from the Institute of Computing Technology, Chinese Academy of Sciences, China, in 2012. He is a Professor and a Ph.D. Supervisor with the School of Cyber Security, University of Chinese Academy of Sciences, China, and the Institute of Information Engineering, Chinese Academy of Sciences, China. His research interests include network and information security, particularly mobile malware attacks and defense.

Tianning Zang received the Master and Ph.D. degrees in computer science from the Harbin Engineering University, China, in 2007 and 2011, respectively. He is currently a senior engineer and Ph.D. supervisor at the Institute of Information Engineering, Chinese Academy of Sciences, China. His research interests include network security, particularly cyberspace security situational awareness.